Project report on

Student Management System with Role-Based Access Control

called as

COMPUS FLOW

Abstract

Student Management System with Role-Based Access Control is a secure, scalable, and multi-module academic management platform designed to digitally transform traditional college operations. The system integrates fourteen independent yet interconnected modules, enabling complete automation of student lifecycle management, academic activities, administrative processes, and communication flows. EduFlow follows a modular micro-app architecture, allowing each functional unit—such as admissions, subject allocation, timetable management, attendance, fees, internal assessment, library services, grievances, and external results—to operate independently while sharing a unified database and authentication mechanism.

The platform incorporates a **role-based access control (RBAC)** and multi-level verification workflow involving **students, faculty, HODs, administrators, principal, and office staff**, ensuring data accuracy, transparency, and compliance. Secure authentication, encrypted data fields, structured API communication, and controlled approval pipelines enhance system reliability and prevent unauthorized access or data tampering. EduFlow also supports automated document management, digital notifications, academic tracking, and semester progression with minimal manual intervention.

By offering an end-to-end digitized academic ecosystem, EduFlow significantly reduces paperwork, processing delays, and human errors, thereby improving administrative efficiency and user experience. The solution is robust, extensible, and designed for future integration with analytics, AI-based predictions, biometric attendance, and mobile application access. Overall, EduFlow aims to modernize institutional management by providing a smart, automated, and workflow-oriented digital campus environment.

Keywords: Django REST Framework, Flutter, RBAC, PostgreSQL, Modular Architecture, Student Information System

Table of Contents

Sl. No	Contents	PageNo.
	Abstract	
1	**Introduction**	
	1.1 Background 1.2 Problems Definition 1.3 Objectives 1.4 Scope 1.5 Unique Features / Innovations 1.6 Target Users 1.7 Expected Outcome	
2	**System Overview**	
	2.1 High-Level Description 2.2 Why Modular (sub-App Based) Architecture 2.3 Advantages of Multi-App Development 2.4 Overall System Block Diagram 2.5 Technology Stack Justification 2.5.1 Backend Framework 2.5.2 Security & Encryption 2.5.3 Utility & API Communication 2.5.4 Time & Scheduling 2.5.5 Database 2.5.6 Networking & Standards 2.5.7 Time & Scheduling	
3	**Architecture & Design**	
	3.1 Overall System Architecture Diagram 3.1.1 Layered View 3.1.2 Security & Authentication Flow 3.2 App-Level Architecture Layout 3.3 Data Flow 3.4 Design Style Used	
4	**Sub-App Documentation**	
	4.1 Sub-App Name: Academics Management App 4.1.1 Purpose & Responsibility 4.1.2 Key Features 4.1.3 Folder / File Structure 4.1.4 Models 4.1.5 API Endpoints 4.1.6 Permissions & Roles 4.1.7 Flow Diagram 4.1.8 Data Flow 4.1.9 Business Rules	

4.1.10 Validations
4.1.11 Error Handling
4.2 Sub-App Name: Student Registration & Profile Management App
4.2.1 Purpose & Responsibility
4.2.2 Key Features
4.2.3 Folder / File Structure
4.2.4 Database Model
4.2.5 API Endpoints
4.2.6 Permissions & Roles
4.2.7 Flow Diagram
4.2.8 Data Flow
4.2.9 Business Rules
4.2.10 Validation Rules
4.2.11 UI Screens
4.1.12 Error Handling
4.3 Sub-App Name: Faculty Management App
4.3.1 Purpose & Responsibility
4.3.2 Key Features
4.3.3 Folder / File Structure
4.3.4 Model
4.3.5 Workflow Logic
4.3.6 API Endpoints
4.3.7 Permissions & Roles
4.3.8 Flow Diagram
4.3.9 Business Rules
4.3.10 Validations
4.3.11 UI Screens
4.3.12 Error Handling
4.4 Sub-App Name: User Login & Authentication Management App
4.4.1 Purpose & Responsibility
4.4.2 Key Features
4.4.3 Folder / File Structure
4.4.4 Database Model
4.4.5 Model Relationships
4.4.6 API Endpoints
4.4.7 Authentication Flow
4.4.8 Password Reset Flow
4.4.9 Business Rules
4.4.10 Validations
4.4.11 UI Screens (Frontend)
4.4.12 Error Handling & Codes

4.5 Sub-App Name: Subject & Faculty Subject Mapping Management App

- 4.5.1 Purpose & Responsibility
- 4.5.2 Key Features
- 4.5.3 Folder / File Structure
- 4.5.4 Database Models
- 4.5.5 API Endpoints
- 4.5.6 Permissions & Access Rules
- 4.5.7 Flow Diagram
- 4.5.8 Business Rules
- 4.5.9 Validations
- 4.5.10 UI Screens (Frontend)
- 4.5.11 Error Handling & Status Codes

4.6 Sub-App Name: Timetable Creation, Approval & Publishing System

- 4.6.1 Purpose & Responsibility
- 4.6.2 Key Features
- 4.6.3 Folder / File Structure
- 4.6.4 Database Models
- 4.6.5 API Endpoints
- 4.6.6 Timetable Creation Workflow
- 4.6.7 Approval Conflict Rules
- 4.6.8 Slot Creation Rules
- 4.6.9 Permissions & RBAC Rules
- 4.6.10 Error Handling

4.7 Sub-App Name: Internal Assessment (IA) Test & Marks Management App

- 4.7.1 Purpose & Responsibility
- 4.7.2 Key Features
- 4.7.3 Folder / File Structure
- 4.7.4 Database Models
- 4.7.5 API Endpoints
- 4.7.6 Permissions & Access Rules
- 4.7.7 Internal Workflow
- 4.7.8 Business Rules
- 4.7.9 Validations
- 4.7.10 UI Screens (Frontend)
- 4.7.11 Error Handling & Status Codes

4.8 Sub-App Name: HOD Assignment & Role Transition Management App

- 4.8.1 Purpose & Responsibility
- 4.8.2 Key Features
- 4.8.3 Folder / File Structure
- 4.8.4 Database Model
- 4.8.5 API Endpoints

4.8.6 Assignment Workflow Logic
4.8.7 Permissions & Access Rules
4.8.8 Business Rules
4.8.9 Validations & Edge Cases
4.8.10 UI / Frontend Screens
4.8.11 Error Handling
4.9 Sub-App Name: Attendance Management & Reporting System
4.9.1 Purpose & Responsibility
4.9.2 Key Features
4.9.3 Folder / File Structure
4.9.4 Database Models
4.9.5 API Endp
4.9.6 Attendance Workflow
4.9.7 Permissions & Access Control
4.9.8 Business Rules
4.9.9 Validations
4.9.10 Error Handling
4.10 Sub-App Name: External Examination Results Upload, Validation & Audit System
4.10.1 Purpose & Responsibility
4.10.2 Key Features
4.10.3 Folder / File Structure
4.10.4 Database Models
4.10.5 Permissions & Access Matrix
4.10.6 PDF Validation Rules
4.10.7 API Endpoints
4.10.8 Workflow (Upload → Validate → Access)
4.10.9 Change Log Workflow
4.10.10 Error Handling & Status Codes
4.10.11 Business Rules
4.11 Sub-App Name: Fees & Payment Management System
4.11.1 Purpose & Responsibility
4.11.2 Key Features
4.11.3 Folder / File Structure
4.11.4 Database Models
4.11.5 API Endpoints
4.11.6 Permission & Roles
4.11.7 Validation Rules
4.11.8 Flow Diagram (High Level)
4.11.9 Data Flow (CRUD)
4.11.10 Business Rules
4.11.11 Error Handling & Status Codes
4.12 Sub-App Name: Grievance Management System
4.12.1 Purpose & Responsibility
4.12.2 Key Features

4.12.3 Folder & File Structure
4.12.4 Models
4.12.5 API Endpoints
4.12.6 Permissions / Role Rules
4.12.7 Flow Diagram
4.12.8 Business Rules
4.12.9 Validations
4.12.10 Error Handling
4.13 Sub-App Name: Library Management System (LMS)
4.13.1 Purpose & Responsibility
4.13.2 Key Features
4.13.3 Folder / File Structure
4.13.4 Database Models
4.13.5 API Endpoints
4.13.6 Permission Matrix
4.13.7 Flow Diagram
4.13.8 Business Rules
4.13.9 Error Handling & Status Codes
4.14 SDMS Admin Panel Configuration Documentation
4.14.1 Custom User & Authentication Admin
4.14.2 Timetable Management Admin
4.14.3 Subject Management Admin
4.14.4 Student Management Admin
4.14.5 Library Management Admin
4.14.6 Internal Assessment Admin
4.14.7 HOD Assignment Admin
4.14.8 Grievance Management Admin
4.14.9 Fees & Accounting Admin
4.14.10 External Results Upload Admin
4.14.11 Attendance Admin
4.14.12 Academics & Master Data Admin
4.15 SDMS Django Configuration Overview
4.15.1 Project Information
4.15.2 Path & Base Directory Setup
4.15.3 Security & Debug Settings
4.15.4 Installed Applications
4.15.5 Middleware Registration
4.15.6 Templates & Rendering
4.15.7 REST Framework Authentication
4.15.8 Database Configuration
4.15.9 Internationalization
4.15.10 Static Files
4.15.11 Custom User Model
4.15.12 Email Configuration (SMTP)
4.15.13 JWT Configuration
4.15.14 URL Routing

5	**Database E-R model**	
6	**Security & Authentication**	
	6.1 Authentication Method 6.2 Role-Based Access Control (RBAC) 6.3 Access Enforcement Locations 6.4 Input Validation & Sanitization Rules 6.5 Sensitive Data Protection 6.6 Major Security Threat Mitigations 6.7 Recommendations for Production	
7	**Full API Reference Guide**	
	7.1. Authentication & User (Module: userlogin) 7.2. Academics Setup (Module: academics) 7.3. Student Management (Module: students) 7.4. Faculty Management (Module: faculty) 7.5. HOD Assignment (Module: hod _assignment) 7. 6. Subject & Assignment Management (Module: subjects) 7.7. Timetable Management (Module: timetable) 7.8. IA Assessment (Module: ia_assessment) 7.9. Attendance (Module: attendance) 7.10. Fees & Payments (Module: fees) 7.11. Grievances (Module: grievances) 7.12. Library Management (Module: library) 7.13. External Results (Module: external_results)	
8	**Testing Documentation**	
	8.1 Test Plan 8.2 Test Case Template 8.2.1 User Authentication & Password Management 8.2.2 Students Module 8.2.3 Faculty + Subject Assignment 8.2.4 Timetable Management 8.2.5 Attendance 8.2.6 IA Assessment 8.2.7 Fees Module 8.2.8 Library Module 8.2.9 Grievances Module 8.3 API Testing Summary 8.4 Unit Testing Summary	
9	**Deployment & Setup Guide**	
	9.1 Prerequisites 9.2 Project Folder Tree 9.3 Environment Variables (.env) 9.4 Install Dependencies 9.5 Database Configuration + Migration 9.5.1 Create database in PostgreSQL 9.5.2 Apply migrations	

	9.5.3 Create admin user 9.6 Running the Backend Server 9.7 Backup Database	
10	**Conclusion**	

1.Introduction

1.1 Background

Educational institutions handle vast amounts of student-related data such as admissions, academics, attendance, assessments, fee transactions, library records, grievances, results, faculty allocation, timetable, and administrative approvals. Traditionally, most institutions use multiple independent software tools, spreadsheets, or partially manual processes, which leads to **data duplication**, **inconsistency**, **slow decision-making**, and **high administrative workload**.

To address these challenges, this project develops an **integrated, role-based, multi-module Student Management System with Role-Based Access Control** using Django REST Framework with secure APIs, modular architecture, authentication, and role-based data access.

1.2 Problem Definition

Current student management processes in many institutions face the following problems:

- Student data spread across multiple applications or maintained manually.
- Lack of centralized system for academic, administrative, and financial modules.
- Difficulty in real-time monitoring, approval, verification, and auditing.
- No automated workflows for timetable, IA assessment, or grievances.
- Complex fee payment tracking and missing receipt verification.
- Limited digital library and book tracking model.
- Poor transparency between students, faculty, HOD, principal, and accounts.

Therefore, there is a need for a **complete, unified, scalable student management ecosystem** that automates daily operations, minimizes human errors, and ensures secure, real-time data accessibility.

1.3 Objectives

Primary Objective

To design and develop a **Student Management System with Role-Based Access Control** that integrates academic, administrative, financial, and communication functionalities under one digital platform.

Specific Objectives

- Provide separate dashboards for students, faculty, HOD, principal, admin, accounts, and library officer.
- Automate workflow and approval processes for timetable, fees, IA, and grievances.
- Maintain verified & accurate student profiles and academic records.
- Generate automated receipts, reports, notifications, and audit trails.
- Ensure secure API communication using token authentication & permission control.

1.4 Scope

This system includes 14 integrated functional sub-modules:

1. User Authentication & Role Management
2. Student Data Management
3. Academic Master Setup
4. Faculty Management
5. HOD Assignment
6. Subject Management & Faculty Mapping
7. Timetable Management & Approval Workflow
8. Attendance Management
9. Internal Assessment (IA / CIE) Marks
10. Fee Structure, Payments, and Receipt Verification
11. Library & E-Library Management
12. Grievance Redressal System
13. External Results Upload & Change Logging

1.5 Unique Features / Innovations

This SDMS system includes several enhancements not found in traditional college portals:

Feature	Description
Multi-role architecture	8+ different user roles with permission-based API
Micro-Module Integration	14 sub-apps working under one system
Automated Workflow	Approval systems for HOD, Principal & Accounts
Fee Workflow Automation	UPI/Bank proof uploads + receipt generation
Audit & Change Log	Tracks who changed what & when
Digital Library	Supports both physical and e-books
Smart Mapping	Subject-Faculty linking with academic schema
Security Layer	Token-based authentication and access control

1.6 Target Users

User Type	Role / Access
Administrator	Full control & master data
Students	Profile, fees, library, IA, results, grievances
Faculty	Attendance, IA marks, timetable
HOD	Department-level approvals and assignments
Principal	Final approvals and monitoring
Accounts	Fee collection, verification & finance
Library Officer	Stock, issue, return, fine & e-library

1.7 Expected Outcome

At the end of implementation, the system will:

- Provide a **single unified portal** for all student operations.
- Reduce **manual paperwork**, duplicate entries, and human errors.
- Improve **data accuracy, transparency, traceability, and reporting**.
- Allow **real-time monitoring and decision making** by stakeholders.
- Increase **student satisfaction** through quick redressal and accessibility.

2.System Overview

2.1 High-Level Description

The **Student Management System with Role-Based Access Control** is a unified, full-stack backend solution designed to digitize, automate, and centralize student-related academic, administrative, and financial operations in an educational institution.

It follows a **REST API-driven, role-based, multi-module architecture** built using Django and Django REST Framework (DRF), enabling seamless data flow across multiple functional domains such as admissions, academics, attendance, fees, IA assessment, library, grievance management, and external results.

The platform exposes modular APIs to be consumed by **mobile applications, web portals (React, Flutter, Angular, etc.), or third-party systems**. It emphasizes **security, scalability, extensibility, and distributed development**.

2.2 Why Modular (Sub-App Based) Architecture

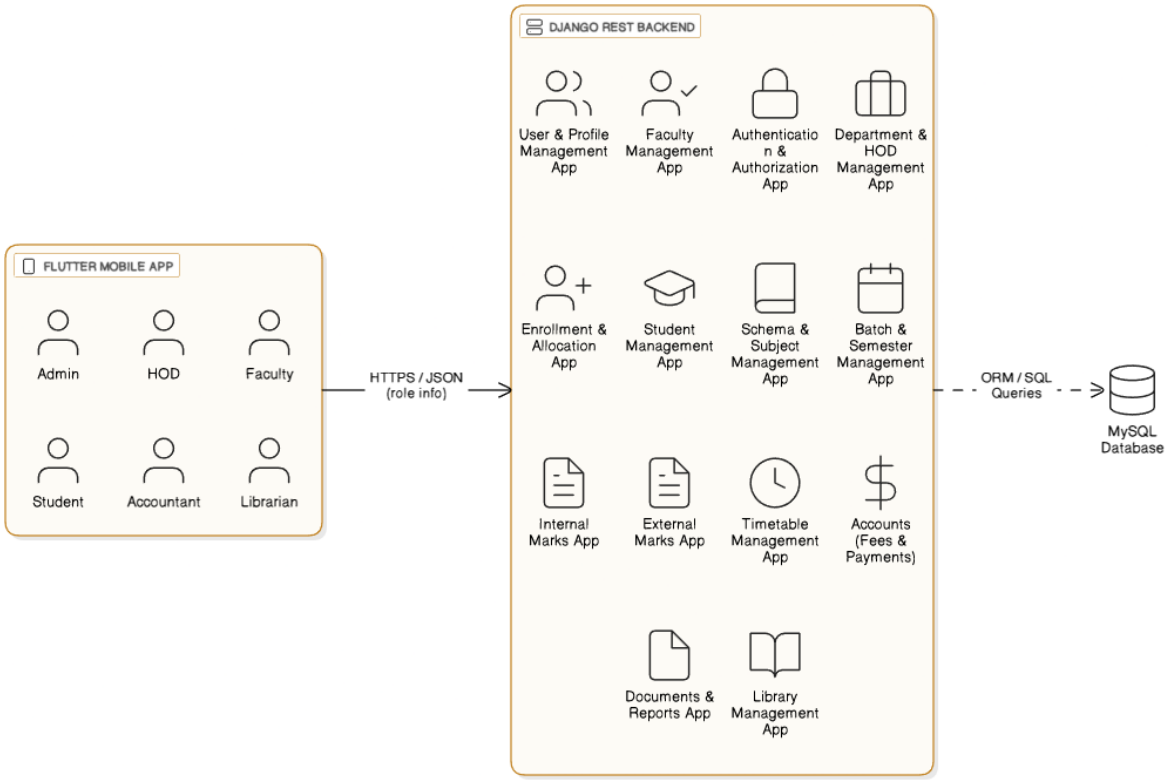
The SDMS platform consists of **14 independent yet interconnected Django apps**. This modular approach ensures:

Reason	Purpose
Separation of Concerns	Each module handles a single responsibility, making code clean and maintainable
Plug-and-Play Modules	New modules can be added without impacting existing ones
Independent Deployment	Services can be containerized & scaled individually
Faster Debugging	Errors can be isolated to their respective modules
Versioning & Reusability	Modules can be reused across other academic systems

2.3 Advantages of Multi-App Development

Advantage	Explanation
Better teamwork	Different developers can work independently on separate modules
Ease of maintenance	Updating one module will not break others
Improved testing	Unit tests & API testing can be done per-module
Enhances security	Access permissions are module-based (ex: Accounts vs Faculty)
Faster feature upgrades	New features can be added without system downtime
Database optimization	Independent DB relationships & indexing per módulo
Scalable architecture	Ready for microservices transition (Docker/K8s)

2.4 Overall System Block Diagram



All modules communicate through secured REST APIs using token-based authentication.

2.5 Technology Stack Justification

2.5.1 Backend Framework

Component	Selection	Reason
Backend Core	**Django 5.2.8**	Robust ORM, secure, production-grade
API Layer	**Django REST Framework (DRF)**	Scalable REST APIs with serializers & permissions
Worker Model	Supports async via Django + ASGI	Ready for WebSockets & async ops

2.5.2 Security & Encryption

Library	Reason
django-fernet-encrypted-fields / django-fernet-fields	Secure encryption of sensitive data
cryptography	Industry-standard cryptographic backend
djangorestframework_simplejwt	Secure token-based authentication

2.5.3 Utility & API Communication

Library	Purpose
requests	External REST API calls
PyMuPDF	PDF handling for receipts & documents
Pillow	Image uploads (photos, proof, docs)

2.5.4 Time & Scheduling

Package	Purpose
asgiref, tzdata	ASGI support & timezone handling

2.5.5 Database

DB	Reason
PostgreSQL + psycopg	Fast transactions, JSON fields, scalable
mysqlclient included	Optional DB migration flexibility

2.5.6 Networking & Standards

Package	Purpose
urllib3, certifi, idna, charset-normalizer	Secure HTTP, SSL certificate & encoding support

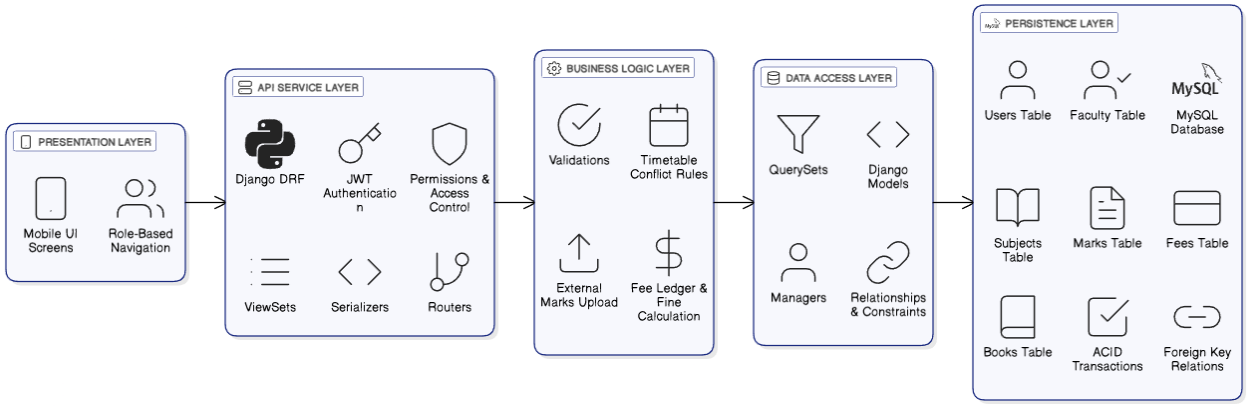
2.5.7 Time & Scheduling

Package	Purpose
asgiref, tzdata	ASGI support & timezone handling

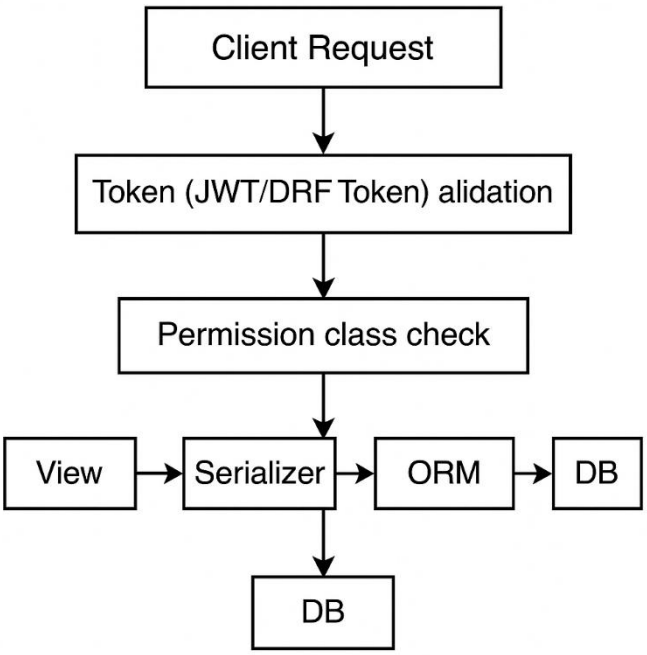
3.Architecture & Design

3.1 Overall System Architecture Diagram

3.1.1 Layered View



3.1.2 Security & Authentication Flow



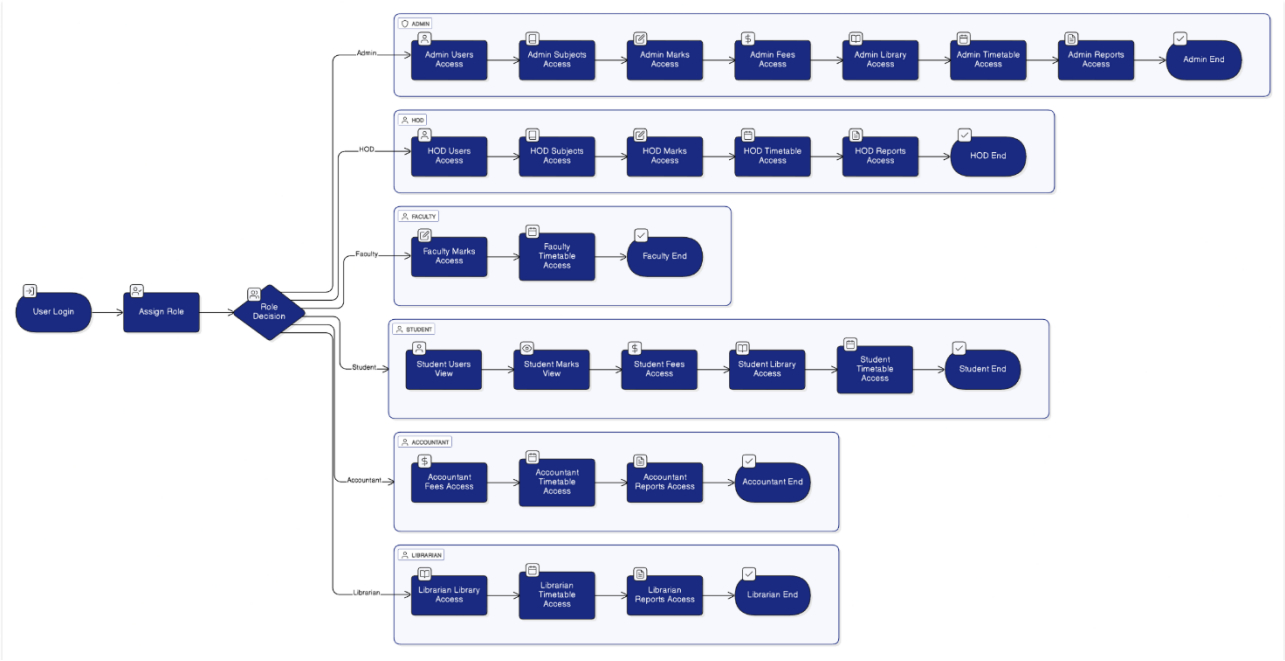
Security controls include:

- Role-based permission classes
- Staff vs student isolation
- Fine-grained object-level permissions
- Encrypted document storage (FERNET)
- Unique constraint validations

3.2 App-Level Architecture Layout

Sub-App Name	Purpose	Depends On	Data Stored	User Roles Involved
userlogin	Authentication, roles, OTP, password mgmt	None	Users table, OTP logs	Admin, All roles
students	Student profile, academic & document records	userlogin, academics	Student KYC, documents, admission data	Admin, HOD, Principal, Student
faculty	Faculty registration, approval & assignment eligibility	userlogin, academics	Faculty profiles	Admin, HOD, Principal
academics	Defines Dept, Semester, Schema & Academic Year	None	Dept, Semester, AY, Schema	Admin, HOD
hod_assignment	Assign faculty as HOD for each dept	faculty, academics	HOD mapping history	Admin, Principal
subjects	Create subjects & assign faculty	academics, faculty	Subject list & faculty mapping	HOD, Faculty, Admin
timetable	Generate timetable & approval flow	subjects, faculty, academics	Timetable & lecture slots	Faculty, HOD, Principal
attendance	Mark & track attendance	students, subjects, faculty	Attendance sessions & logs	Faculty, HOD, Student
fees	Fee structure, payments, receipts	students, academics	Fee structure, receipts, proofs	Accounts, Principal, Students
ia_assessment	IA tests, marks & aggregates	students, subjects, faculty	IA marks & analytics	Faculty, HOD, Student
library	Books, copies, issuance, fines, ebook mgmt	academics, userlogin	Books, fines, e-book links	Library Officer, Student, Faculty
grievances	Lodge & handle grievances	students, userlogin	Complaints & responses	Student, Faculty, HOD, Principal, Accounts
external_results	Upload & verify external exam PDF	students, academics	Result PDF, changelog	Student, HOD

3.3 Data Flow



3.4 Design Style Used

Category	Design Decision
Architecture Pattern	Modular Monolithic (Microservice-ready)
API Style	REST JSON
Security	JWT + Token Authentication
Database	Normalized Relational (PostgreSQL)
Code Pattern	Model → Serializer → ViewSet → URL
Frontend Interaction	Stateless API consumption

4. Sub-App Documentation

4.1 Sub-App Name: Academics Management App

4.1.1 Purpose & Responsibility

The Academics Management App handles the core academic structural data required throughout the student management system. It manages academic entities such as **Departments**, **Semesters**, **Academic Years**, and **Schemas (Curriculum Plans)**, which will be linked with subjects, faculty assignment, and student enrollment across modules.

4.1.2 Key Features

- Create, list, update, and delete departments.
- Manage semester records (1 to N).
- Manage academic year cycles like 2025-2026.
- Create schemas/curriculum linked to departments.
- Filter schemas by department.
- Public read access for some endpoints, restricted modifications.

4.1.3 Folder / File Structure

```
backend/
└── academics/
    ├── models.py
    ├── serializers.py
    ├── views.py
    ├── urls.py
    ├── apps.py
    ├── admin.py
    ├── test.py
```

4.1.4 Models

Department

Table Name: department

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
name	CharField(100)	Department Name	Unique, Required

Semester

Table Name: semester

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
sem_number	IntegerField	Semester number (1 to N)	Default = 1

Schema

Table Name: schema

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
name	CharField(100)	Schema/Curriculum name	Unique
department	ForeignKey(Department)	Linked Department	Nullable, SET_NULL, related_name="schemas"

AcademicYear

Table Name: academic_year

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
year_name	CharField(20)	Format: 2025-2026	Unique, Required

Relationships

- **Department → Schema**: One-to-Many
- No cascade delete to protect data integrity

4.1.5 API Endpoints

Method	URL	Purpose	Permissions
GET	/api/academics/departments/	List departments	Public
POST	/api/academics/departments/	Create department	Admin
GET	/api/academics/departments//	Retrieve	Public
PUT	/api/academics/departments//	Update	Admin
DELETE	/api/academics/departments//	Delete	Admin
GET	/api/academics/semesters/	List semesters	Public
POST	/api/academics/semesters/	Create semester	Admin
GET	/api/academics/semesters//	Retrieve	Public
PUT	/api/academics/semesters//	Update	Admin
DELETE	/api/academics/semesters//	Delete	Admin
GET	/api/academics/academic-years/	List years	Authenticated
POST	/api/academics/academic-years/	Create year	Authenticated
GET	/api/academics/academic-years//	Retrieve	Authenticated
PUT	/api/academics/academic-years//	Update	Authenticated
DELETE	/api/academics/academic-years//	Delete	Authenticated
GET	/api/academics/schemas/?department=	List schemas	Public
POST	/api/academics/schemas/	Create schema	Authenticated
GET	/api/academics/schemas//	Retrieve	Authenticated
PUT	/api/academics/schemas//	Update	Authenticated
DELETE	/api/academics/schemas//	Delete	Authenticated

4.1.6 Permissions & Roles

Action	Role
List Departments/Semesters/Schemas	Public
Create/Update/Delete Departments & Semesters	Admin
Create/Update/Delete Academic Years & Schemas	Authenticated

4.1.7 Flow Diagram

Request → Permissions Check → Validation → Serializer → DB Action → Response

4.1.8 Data Flow

Admin creates department → Schema created under it → Subjects mapped → Faculty & student operations depend on schema.

4.1.9 Business Rules

- Department name must be unique
- Schema name must be unique
- Academic Year must follow recognized format (YYYY-YYYY)
- Semester can only be integer values
- Schema cannot be deleted if subjects or faculty mapping exist (future constraint)

4.1.10 Validations

- Required fields check
- Unique check
- Integer format validation
- Max-length constraints

4.1.11 Error Handling

Code	Description
200	Success
201	Created
400	Invalid data
401	Unauthorized
403	Forbidden
404	Not Found
409	Duplicate Value
500	Server Error

4.2 Sub-App Name: Student Registration & Profile Management App

4.2.1 Purpose & Responsibility

This app manages complete student registration, personal profile storage, academic linkage, identity creation, document uploads, semester promotion, verification workflow, and approval-based activation.
It securely stores sensitive student data using encrypted fields and integrates with Django CustomUser for login credentials.

4.2.2 Key Features

- Online student registration form (public access)
- Auto-generate unique Admission ID (GECC000001 format)
- Create linked CustomUser with temporary password
- Store encrypted personal, contact, and parental details
- Upload certificates, marksheets, photos & signature
- Multi-step approval workflow:
    1. Student Registration
    2. Department Verification
    3. Principal Approval
- USN assignment
- Semester promotion automation
- Student self-profile view
- Prevent login until final approval
- Academic year auto-rollover for odd semesters

4.2.3 Folder / File Structure

```
backend/
└── students/
    ├── app.py
    ├── admin.py
    ├── test.py
    ├── models.py
    ├── serializers.py
    ├── views.py
    ├── urls.py
    ├── permissions.py
    ├── utils.py (email sending)
└── templates/email/student_approval.html
```

4.2.4 Database Model

Table Name: student_details

Field	Type	Description	Constraints
id	AutoField	PK	PK
user	FK → CustomUser	Login user relation	OneToOne, Cascade
name	CharField(100)	Full name	Required
cet_no	CharField(50)	CET Number	Required
dob	DateField	Date of birth	Required
gender	ChoiceField	Male/Female/Other	Optional
religion	EncryptedCharField	Encrypted	Required
city_type	EncryptedCharField	Urban/Rural	Required
mobile_no	EncryptedCharField(15)	Phone	Unique (via User)

email	EncryptedEmailField	Email	Unique
father_name/mother_name	EncryptedCharField	Encrypted	Optional
parent_occupation	EncryptedCharField	Encrypted	Required
address/permanent_address	EncryptedTextField	Full address	Required
nationality	EncryptedCharField	Country	Optional
state	EncryptedCharField	State	Optional
date_of_admission	DateField	Admission date	Required
seat_type	ChoiceField	GM/SNQ/Lateral	Required
other_reservation	EncryptedCharField	Reservation type	Optional
caste	EncryptedCharField	Community	Optional
department	FK → Department	Academic mapping	Nullable
semester	FK → Semester	Academic mapping	Nullable
academic_year	FK → AcademicYear	Academic mapping	Nullable
admission_id	CharField(20)	Unique format GECC000001	Auto-generated
usn	CharField(20)	Unique student number	Optional
admission_quota	CharField(100)	MQ/KEA/etc.	Optional
rank	EncryptedCharField(20)	Ranking data	Optional
status	ChoiceField	Active/Inactive	Optional
registration_status	ChoiceField	pending/verified/approved/rejected	Default=pending
sslc/puc/diploma*	FileFields	Document uploads	Optional
certificates	FileFields	Category certificates	Optional
photo/signature	ImageField	Identity	Optional
temporary_password	CharField	System generated password	Viewable once

Model Relationships

- **Student → CustomUser**: One-To-One
- **Student → Department**: Many per Department (FK)
- **Student → Semester**: Many per Semester (FK)
- **Student → AcademicYear**: Many per Year (FK)

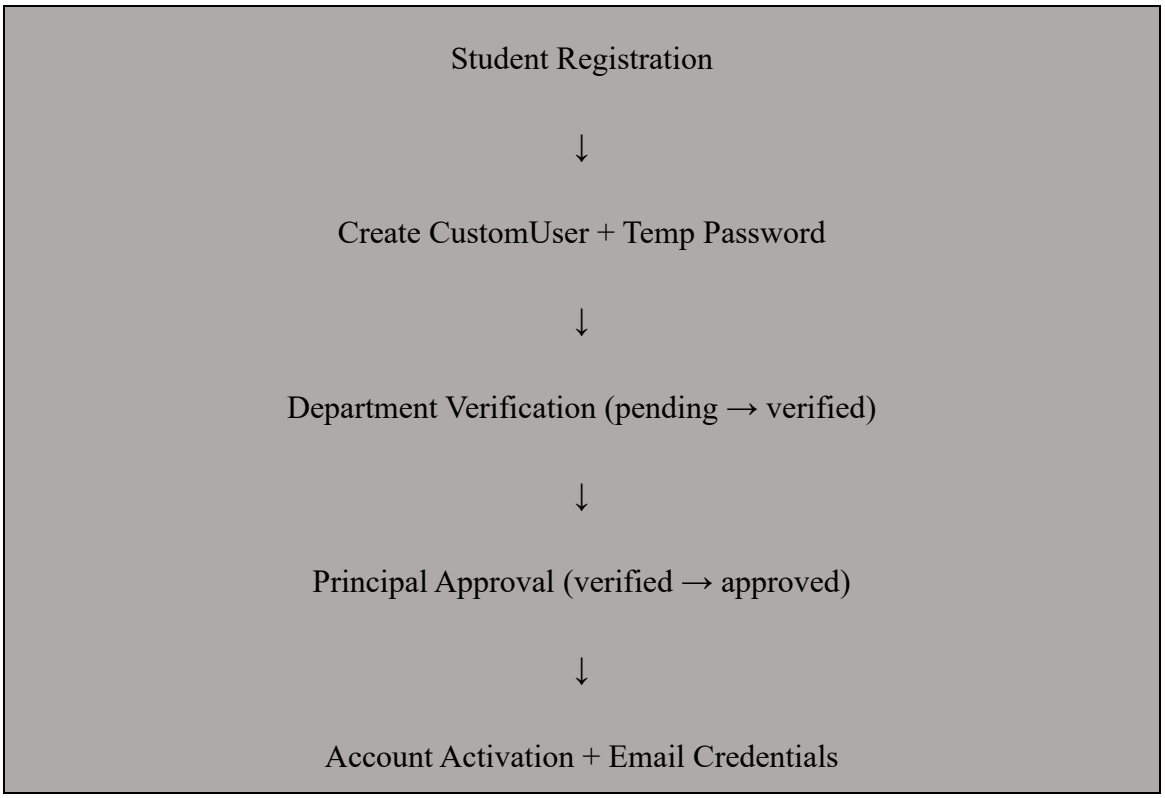
4.2.5 API Endpoints

Method	URL	Purpose	Permissions
POST	/api/students/create/	Student registration	Public
GET	/api/students/list/	List filtered students	Authenticated / HOD restricted
GET/PUT	/api/students//	View/Update student	Authenticated
PUT	/api/students//assign-usn/	Assign USN	Authenticated
PUT	/api/students//verify/	Department verification	Authenticated (Dept role)
PUT	/api/students//approve/	Principal approve/reject	Authenticated (Principal role)
GET	/api/students/me/	Student self profile	Authenticated
PUT	/api/students//promote/	Promote semester	HOD only

4.2.6 Permissions & Roles

Action	Role
Registration	Public
Profile View	Owner Only
Department Verification	Department Staff
Principal Approval	Principal Only
Promote Student	HOD
List All	HOD Only

4.2.7 Flow Diagram



4.2.8 Data Flow

Input → Validate → Create User → Create Student → Auto Admission ID → Response Output → Approval Flow

4.2.9 Business Rules

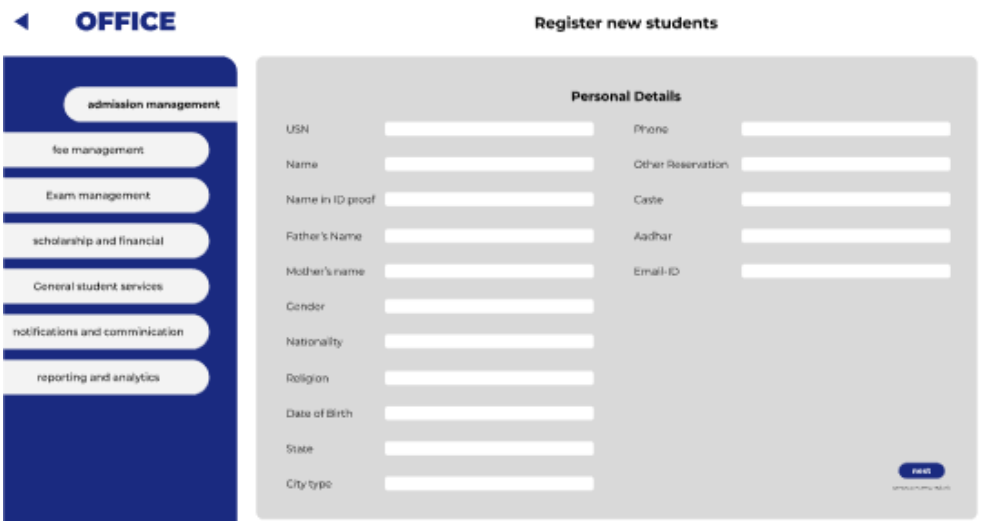
- Email & Phone must be unique (checked in User model)
- Admission ID auto-generated and never reused
- Student cannot log in until approval
- USN assignable only once and must be unique
- Semester promotion stops after sem 8 and deactivates student

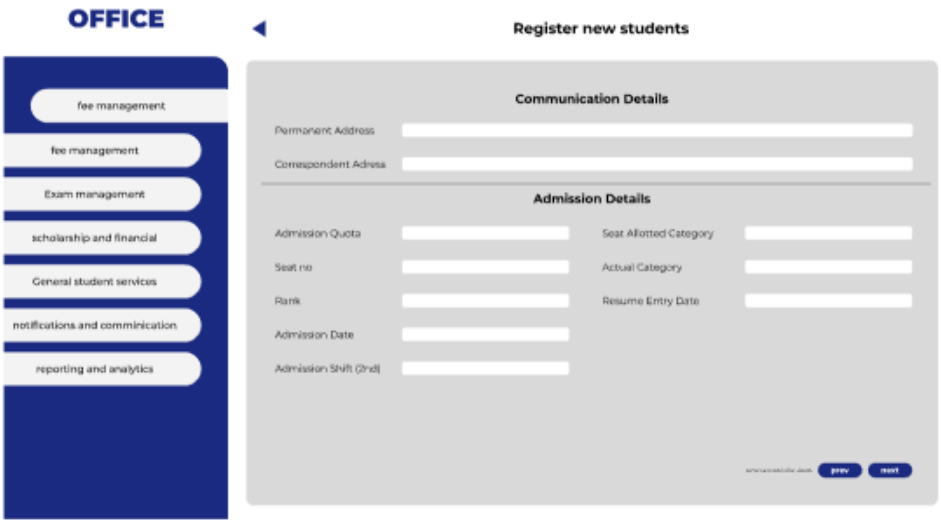
4.2.10 Validation Rules

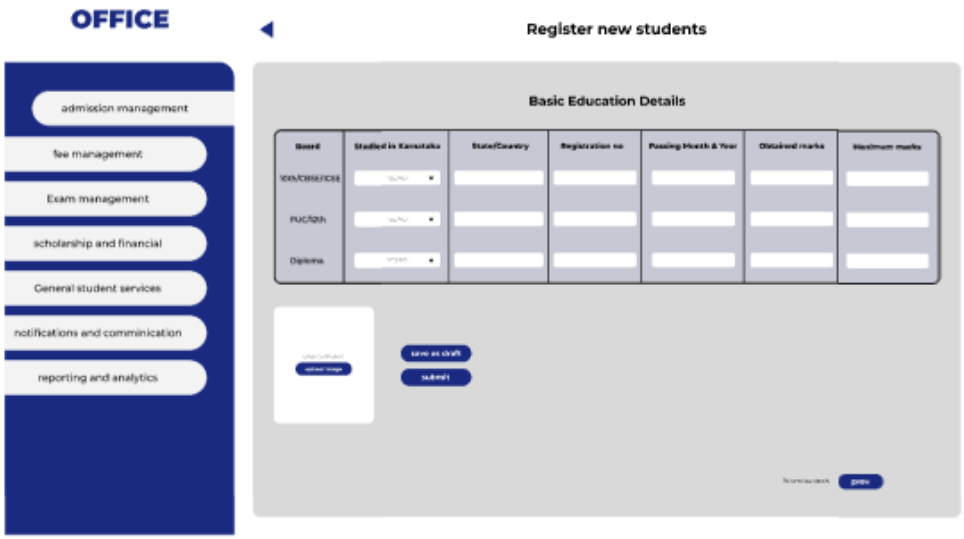
- Required fields: name, email, mobile_no, seat_type, date_of_admission
- Encrypted fields applied for personal and sensitive data
- Semester must be assigned for promotion
- File type validation required (future task)

4.2.11 UI Screens

- Student Registration Form







- Department Verification Dashboard

- Principal Approval Panel
- Student Profile View (Read Only)
- Semester Promotion Panel (HOD)

4.2.13 Error Handling

Status	**Description**
200	Success
201	Created
400	Invalid / Missing Data
401	Unauthorized
403	Permission Denied
404	Student Not Found
409	Duplicate Data
500	Internal Server Error

4.3 Sub-App Name: Faculty Management App

4.3.1 Purpose & Responsibility

The Faculty Management App handles faculty onboarding, identity creation, encrypted profile storage, department-based role assignment, workflow approval stages, account creation with credentials, and automatic principal auto-approval logic. It integrates with CustomUser for authentication and enforces strict security rules using encrypted fields and controlled approval workflow.

4.3.2 Key Features

- Create & update faculty profiles
- Auto-generate login account upon approval
- Secure & encrypted personal data
- Workflow: Office Verification → Principal Approval
- Auto-approval for Principal role
- Auto-unique principal enforcement (only one principal allowed)
- Auto email notifications with temporary password
- REST API using ModelViewSet
- Role mapping via predefined choices
- Real-time approved date tracking

4.3.3 Folder / File Structure

```
backend/
└── faculty/
    ├── app.py
    ├── admin.py
    ├── test.py
```

```
├── models.py
├── serializers.py
├── views.py
├── urls.py
├── signals.py
├── permissions.py
├── utils.py (email sending)
└── templates/email/faculty_approval.html
```

4.3.4 Model

Table Name: faculty

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
user	OneToOne FK → CustomUser	Linked system user	Nullable
faculty_id	CharField(50)	Official faculty ID	Unique, Required
name	CharField(200)	Full name	Required
gender	ChoiceField	Male/Female/Other	Optional
dob	DateField	Date of birth	Optional
address	EncryptedTextField	Residential address	Optional, Encrypted
phone	EncryptedCharField (15)	Contact number	Unique
email	EncryptedEmailFiel d	Faculty email	Unique
department	FK → Department	Assigned Department	Required
qualification	CharField(200)	Highest qualification	Optional
role	ChoiceField	Principal / HOD / Professor / etc.	Required
status	ChoiceField	Workflow status	Default=pending_off ice
registration_date	DateTimeField	Registered timestamp	Auto
approved_date	DateTimeField	Approval timestamp	Auto when approved

Model Relationships

Relationship	Type
Department → Faculty	One-to-Many
CustomUser → Faculty	One-to-One

4.3.5 Workflow Logic

Stage	Description	Controlled By
pending_office	Submitted but not verified	Office staff
pending_principal	Waiting for principal approval	Principal
approved	Auto-create User + Email login	System
rejected	Disabled from system use	Admin/Principal

- Additional Logic
  - Principal role is auto-approved (pending_principal gets changed to approved)
  - Only one principal can exist (others lose principal role automatically)

4.3.6 API Endpoints

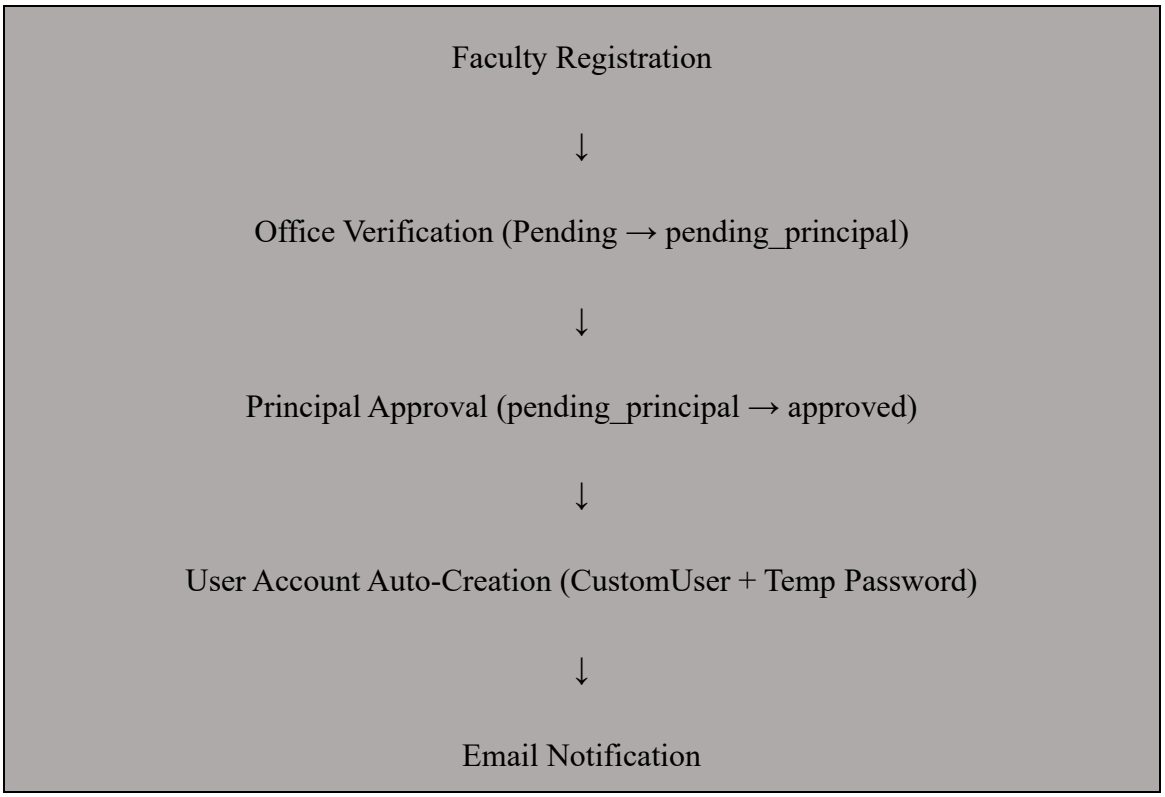
This app uses **ModelViewSet with router**, hence full CRUD:

Method	URL	Purpose
GET	/api/faculties/	List all faculty
POST	/api/faculties/	Create new faculty
GET	/api/faculties/{id}/	View faculty profile
PUT	/api/faculties/{id}/	Update profile
PATCH	/api/faculties/{id}/	Partial update
DELETE	/api/faculties/{id}/	Delete

4.3.7 Permissions & Roles

Action	Allowed Roles
Create Faculty	Admin / Office
Update Faculty	Admin / Office
Approve Faculty	Principal
Auto approve	Principal-role users
Delete Faculty	System Admin

4.3.8 Flow Diagram



4.3.9 Business Rules

- faculty_id, phone, email must be unique
- Only one principal active at a time
- Email credentials sent only after approval
- Updating principal automatically resets previous principal role
- user record auto-created only when status becomes approved

4.3.10 Validations

- Email pattern must be valid
- Phone must contain digits only
- Role must be from predefined list
- Department cannot be null
- Status change validation required

4.3.11 UI Screens

- Faculty Registration Page
- Verification Dashboard (Office)
- Principal Approval Dashboard
- Approved Faculty Directory Page
- Faculty Profile Viewer

4.3.12 Error Handling

Status	Meaning
200	Success
201	Created
400	Invalid data
401	Unauthorized
403	Permission denied
404	Faculty not found
409	Duplicate value conflict
500	Server error

4.4 Sub-App Name: User Login & Authentication Management App

4.4.1 Purpose & Responsibility

This sub-app manages **user accounts, authentication, roles, login access, OTP-based password reset system, JWT token-based authentication, and secure role-based authorization**.

It provides a unified login mechanism for **all system actors**: Students, Faculty, HOD, Principal, Accounts, Exam Cell, Library, Scholarship, and System Admin.

4.4.2 Key Features

- Custom user model (supports login using email / phone / user ID)
- JWT login & token refresh
- Role-based access control system (RBAC)
- Forced password change for first login
- Password encryption using Django hashing
- OTP-based secure password reset
- Admin-controlled direct password reset
- Tracks last seen and date joined
- Multi-identifier login support

4.4.3 Folder / File Structure

```
backend/
└── userlogin/
    ├── apps.py
    ├── admin.py
    ├── test.py
```

```
├── models.py
├── serializers.py
├── views.py
├── utils.py
├── urls.py
└──templates/email/otp_template.html
```

4.4.4 Database Model

Table: custom_user

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
email	EmailField	Login email	Unique, Nullable
phone	CharField(15)	Mobile number	Unique, Nullable
user_id	CharField(50)	Custom system ID	Unique, Nullable
usn	CharField(20)	Student USN	Unique, Nullable
role	ChoiceField	User role	Default = STUDENT
department	FK → Department	User's department	Optional
force_password_change	Boolean	Force change on next login	True by default
is_active	Boolean	Account status	True by default
is_staff	Boolean	Admin panel access	False
date_joined	DateTimeField	Account creation timestamp	Auto
last_seen	DateTimeField	Recent logged-in timestamp	Optional

Table: password_reset_otp

Field	Type	Description	Notes
user	FK → CustomUser	Related user	Cascade
otp	CharField(6)	One-time password	Auto generated
created_at	DateTimeField	OTP timestamp	Used for expiry check

OTP Expiry Rule: 10 minutes

Method: is_expired()

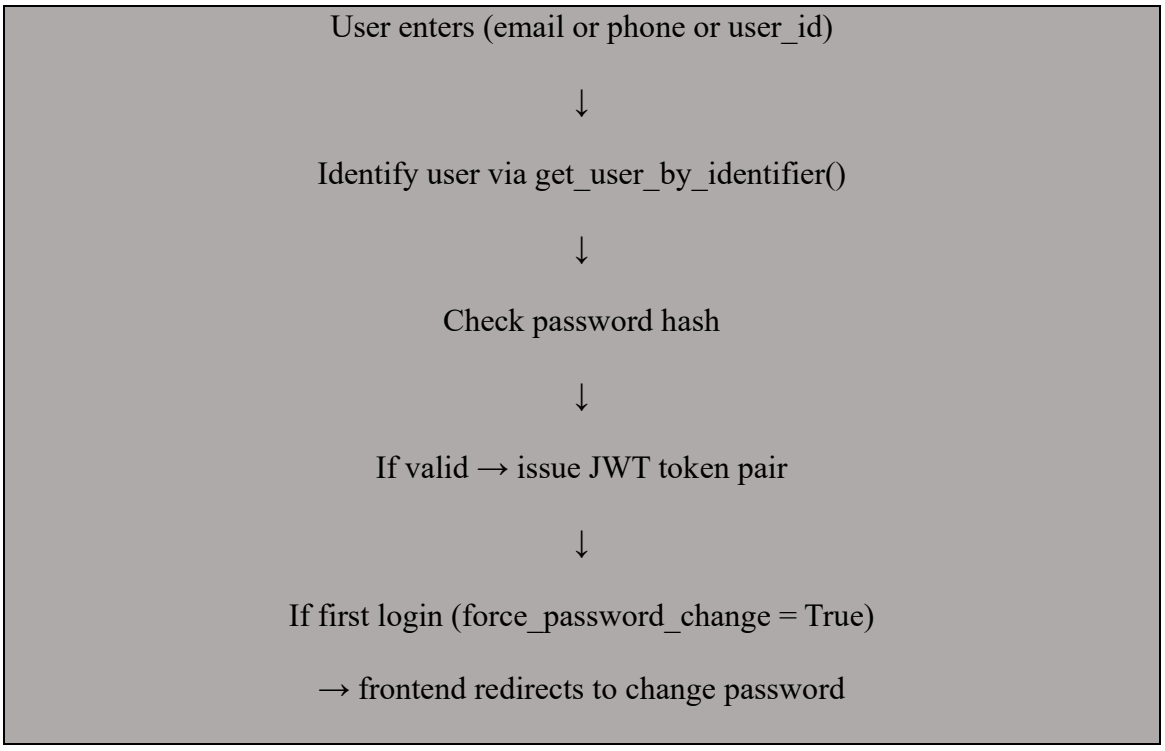
4.4.5 Model Relationships

Relationship	Type
CustomUser → Department	Many users per department
CustomUser → PasswordResetOTP	One-to-many

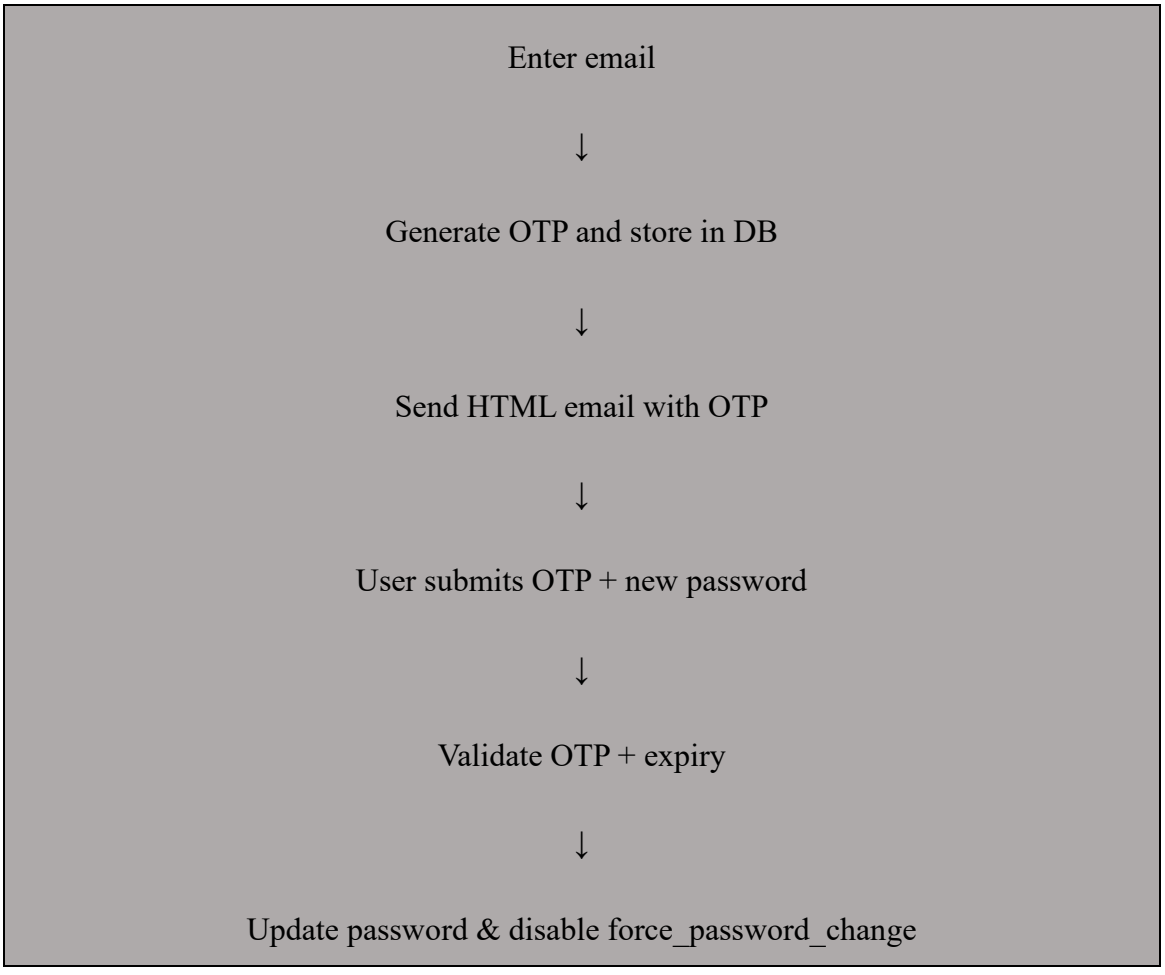
4.4.6 API Endpoints

Method	URL	Purpose	Permissions
POST	/api/userlogin/login/	Authenticate user via email/phone/userID	Public
POST	/api/userlogin/password/forgot/	Request OTP	Public
POST	/api/userlogin/password/verify/	Verify OTP + Reset Password	Public
POST	/api/userlogin/password/change/	Change Password	Authenticated
POST	/api/userlogin/password/reset/	Admin direct reset	Admin Only
POST	/api/userlogin/token/refresh/	Refresh Access Token	Authenticated

4.4.7 Authentication Flow



4.4.8 Password Reset Flow



4.4.9 Business Rules

- At least **one login identifier** must be provided (email/phone/user_id)
- Email and phone must be **unique** (but optional per user)
- OTP expires in **10 minutes**
- Duplicate emails are not allowed
- force_password_change = True after system-generated passwords
- System Admin accounts can reset password **without OTP**
- Role-based access determines UI navigation

4.4.10 Validations

Rule	Validation
Identifier required	email or phone or user_id
Password format	Handled by frontend or policy
OTP validity	6 digits + expiry check
User existence	Case insensitive lookup

4.4.11 UI Screens (Frontend)

- Login Screen (multi identifier)
- Forgot Password (OTP request)
- OTP Verification Modal
- Change Password Screen
- Admin Password Reset Dashboard

4.4.12 Error Handling & Codes

Code	Meaning

200	Success
201	OTP generated
400	Missing/Invalid input
401	Unauthorized
403	Insufficient role permission
404	User not found
409	Duplicate email/phone
500	Internal server error

4.5 Sub-App Name: Subject & Faculty Subject Mapping Management App

4.5.1 Purpose & Responsibility

This sub-app manages **academic subjects and their mapping to faculty members for a specific department, semester, and academic year**.
It supports subject creation, listing, dynamic filtering, and assignment operations, ensuring that each subject is properly handled and assigned to the relevant teaching staff with constraints to prevent duplication.

4.5.2 Key Features

- Add new subjects under department, semester, and academic schema.
- Retrieve subject lists with optional search & filters.
- Assign multiple subjects to faculty in bulk for a specific academic year.
- Prevent duplicate subject–faculty–year–department–semester mappings.
- Retrieve assigned faculty based on filters.
- View assignments department-wise and faculty-wise.

4.5.3 Folder / File Structure

```
backend/
└── subjects/
    ├── apps.py
    ├── admin.py
    ├── test.py
    ├── models.py
    ├── serializers.py
    ├── views.py
    ├── urls.py
```

4.5.4 Database Models

Model: Subject

Table Name: subjects_subject

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
schema	FK → Schema	Academic scheme/curriculum	Nullable
department	FK → Department	Academic department	Required
semester	FK → Semester	Semester to which subject belongs	Required
subject_name	CharField(200)	Name of the subject	Required
subject_code	CharField(50)	Unique code	Unique, Optional

Relationships

- Schema → Subject : One-to-Many
- Department → Subject : One-to-Many
- Semester → Subject : One-to-Many

Model: SubjectAssignment

Table Name: subjects_subjectassignment

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
subject	FK → Subject	Subject to teach	Required
faculty	FK → Faculty	Assigned faculty member	Required
academic_year	FK → AcademicYear	Year for which assignment is active	Required
department	FK → Department	Responsible department	Required
semester	FK → Semester	Semester assignment applies to	Nullable

- **Unique Constraint**

unique_together = (subject, faculty, academic_year, department, semester)

Prevents duplicate allocation for same year & class.

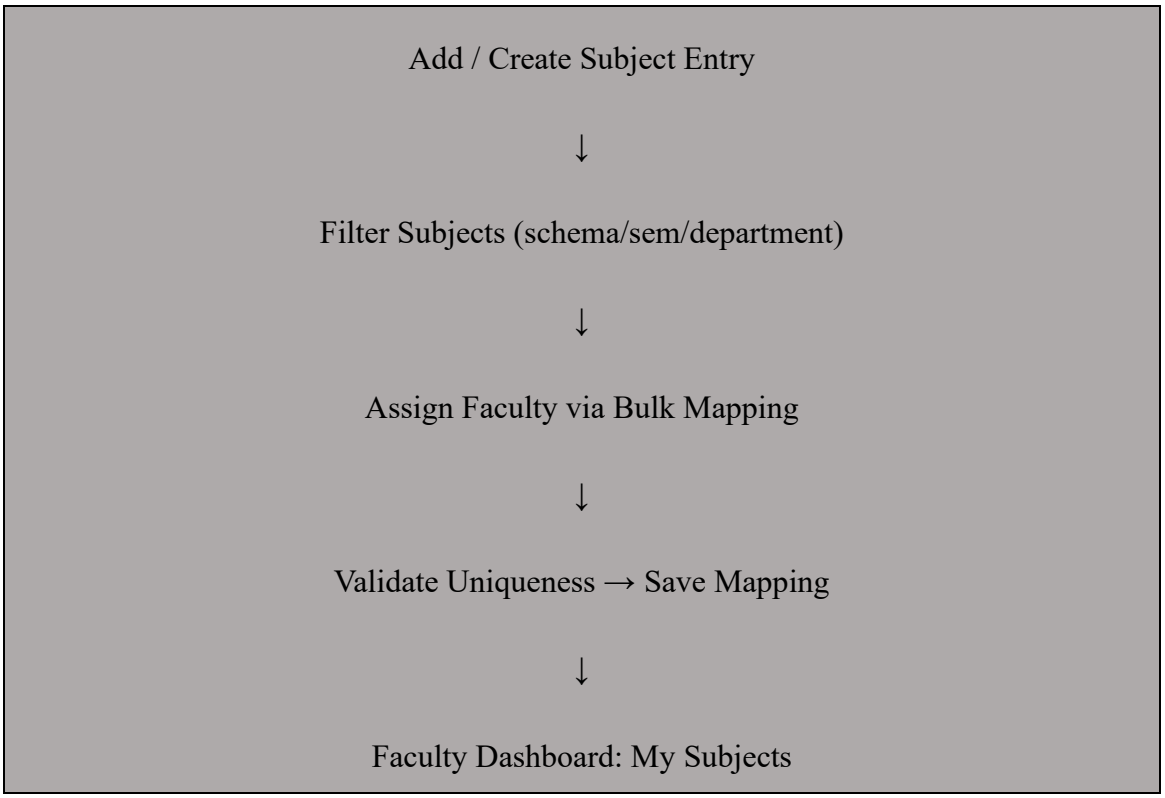
4.5.5 API Endpoints

Method	URL	Purpose	Permission
POST	/api/subjects/addSubject/	Add new subject	Authenticated
GET	/api/subjects/getSubjects/	List subjects with optional filters	Authenticated
POST	/api/subjects/assign_subjects/	Bulk assign faculty	Authenticated
GET	/api/subjects/getAssignedProfessors/	Get faculty list teaching subjects	Authenticated
GET	/api/subjects/view-assigned-by-department/	Department-wise subject mapping	Authenticated
GET	/api/subjects/professor/subjects/	Assigned subjects for faculty	Authenticated

4.5.6 Permissions & Access Rules

User Type	Access
Admin / HOD	Full CRUD + Assign subjects
Faculty	Can view own assigned subjects
Student	No access
System/Library/Exam/Accounts	No access

5.5.7 Flow Diagram



4.5.8 Business Rules

- subject_code must be unique across the college.
- A subject cannot be assigned twice for the same:
  (subject + faculty + year + dept + semester)
- Faculty can be assigned subjects only within valid dept/year.
- Subject must exist before assignment.
- Valid academic year must be selected.

4.5.9 Validations

- All required fields must be present in assignment payload.
- Subject existence and faculty existence validation handled before saving.
- Disallow missing keys in bulk assignment loop.
- Semester matching constraints must be respected.

4.5.10 UI Screens (Frontend)

- Add Subject Page (Admin/HOD)
- Subject List with department & semester filters
- Assign Faculty (Bulk grid selection)
- View Assigned Subjects (HOD/Department Panel)
- Faculty Self Dashboard – *My Subjects*

4.5.11 Error Handling & Status Codes

Status	Meaning
200	Success
201	Created
400	Missing fields / duplicate data
401	Unauthorized
403	Permission denied
404	No records found
500	Internal server error

4.6 Sub-App Name: Timetable Creation, Approval & Publishing System

4.6.1 Purpose & Responsibility

This sub-app manages **academic timetables** including creation, versioning, verification, conflict detection, approval workflow, publishing, and viewing by stakeholders (HOD, Principal, Faculty, Students).
It ensures **schedule accuracy**, **faculty workload validation**, and **version control** for each academic year, department, and semester.

4.6.2 Key Features

- Create and version timetables automatically.
- Add multiple timetable slots with subject-faculty mapping.
- Prevent overlapping faculty class conflicts.
- Draft → Pending → Approved/Rejected workflow.
- Auto-deactivate previous published version upon approval.
- Track approval history with remarks and timestamps.
- Role-based filtered viewing:
    - HOD: Manages creation
    - Principal: Approves/rejects timetables
    - Professors: View assigned classes
    - Students: View class timetable for their semester

4.6.3 Folder / File Structure

```
backend/
└── timetable/
    ├── apps.py
    ├── models.py
    ├── admin.py
    ├── test.py
    ├── serializers.py
    ├── views.py
    ├── permissions.py
    ├── urls.py
```

4.6.4 Database Models

Model: TimeTable

Table Name: timetable_timetable

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
department	FK → Department	Department timetable belongs to	Required
semester	FK → Semester	Academic semester	Required
academic_year	FK → AcademicYear	Timetable academic cycle	PROTECT
version	PositiveIntegerField	Auto-increment per dept+sem+year	Default = 1
created_by	FK → CustomUser	Created user	Nullable
created_at	DateTimeField	Creation timestamp	Auto
status	ChoiceField	Draft/Pending/Approved/Rejected	Default = Draft
is_active	Boolean	Indicates published state	Default = False

approved_by	FK → CustomUser	Principal approval	Nullable
approved_at	DateTimeField	Approval timestamp	Nullable
remarks	TextField	Rejection/approval comment	Optional

Automatic Versioning Rules

- New timetable increments highest version found under same **department + semester + academic year**
- Previous active timetables will be auto-disabled upon approval

Model: TimeTableSlot

Table Name: timetable_timetableslot

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
timetable	FK → TimeTable	Parent timetable	CASCADE
day	ChoiceField	Mon-Sat	Required
start_time	TimeField	Class start time	Required
end_time	TimeField	Class end time	Must be > start_time
assignment	FK → SubjectAssignment	Lecturer-subject pair	Required

Unique Constraint

unique_together = (timetable, day, start_time, end_time, assignment)

Validation Rules

- No faculty overlapping allowed within approved timetables
- start_time < end_time

Model: TimeTableApprovalHistory

Tracks all decisions (audit log)

Field	Type	Description
timetable	FK → TimeTable	Target timetable
action	ChoiceField	Approved/Rejected
remarks	TextField	Optional
action_by	FK → CustomUser	Who performed
action_at	DateTimeField	Timestamp

4.6.5 API Endpoints

Using DRF ModelViewSet → /api/timetable/timetables/

Method	URL	Action	Role
GET	/timetables/	List all timetables	Authenticated
POST	/timetables/	Create timetable	HOD only
GET	/timetables/{id}/	Retrieve timetable	Authenticated
PUT/PATCH	/timetables/{id}/	Update timetable	HOD
DELETE	/timetables/{id}/	Delete timetable	HOD

Custom Actions

Method	URL	Purpose	Role
GET	/timetables/assignments/	Get allowed subject-faculty list	HOD
POST	/timetables/{id}/approve/	Approve or reject	Principal
GET	/timetables/pending/	List timetables (Pending state)	Principal
GET	/timetables/faculty/	Get active timetable for faculty	Professor
GET	/timetables/student/	Get timetable for logged-in student	Student
GET	/timetables/hod/	Get timetable for department	HOD

4.6.6 Timetable Creation Workflow

```
HOD Creates Timetable → Add Slots → Submit → Status: Pending
                ↓
Principal Reviews Timetable
                ↓
Approved? → Yes → Publish (is_active=True)
        |
        └── No → Return to HOD with Remarks (Rejected)
```

4.6.7 Approval Conflict Rules

Before approval, system checks:

Same faculty

Same day

Overlapping time range

Active timetable exists

If **conflict detected** → Response 409 CONFLICT with full conflict details Principal can override using "override": true

4.6.8 Slot Creation Rules

- Assigned subject must belong to same department+semester+academic_year
- No slot without subject assignment
- Time must not overlap inside same timetable

4.6.9 Permissions & RBAC Rules

Role	Access
HOD	Create, edit, request approval, view
Principal	Approve/reject, view pending
Professor	View own active timetable
Student	View assigned semester timetable
Others	Denied

4.6.10 Error Handling

Status	Error
400	Invalid or missing slot details
401	Unauthorized
403	Not allowed for role
404	Timetable not found
409	Slot conflict detected
500	Server runtime error

4.7 Sub-App Name: Internal Assessment (IA) Test & Marks Management App

4.7.1 Purpose & Responsibility

This sub-application manages **internal assessments (IA)** including test creation, student mark entry, updating, aggregation, reporting, and access control for **faculty and students**.

It supports multiple types of assessments such as **Unit Tests, Assignments, and Quizzes** and calculates **per-subject totals and averages automatically**.

4.7.2 Key Features

- Create IA test setup for each subject.
- Multiple IA types supported (Test1, Test2, Assignment, Quiz).
- Fetch only active and valid students dynamically.
- Bulk mark entry and update.
- Auto recalculation of subject-wise totals and averages.
- Faculty-specific access to IA data.
- Students can view only their own IA marks.
- Duplicate prevention per subject/faculty/test combination.

4.7.3 Folder / File Structure

```
backend/
└── ia_assessment/
    ├── apps.py
    ├── admin.py
    ├── test.py
    ├── models.py
```

```
├── serializers.py
├── views.py
├── urls.py
```

4.7.4 Database Models

IATestSetup

Table Name: ia_assessment_iatestsetup

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
subject	FK → Subject	Subject under assessment	Required
faculty	FK → User	Responsible faculty	Required, Auto-set
ia_type	ChoiceField	Test1/Test2/Assignment/Quiz	Required
ia_name	CharField(100)	Label (e.g., “Unit Test 1”)	Required
max_marks	PositiveIntegerField	Maximum possible marks	Required
created_at	DateTimeField	Entry timestamp	Auto

Unique Constraint

(subject, faculty, ia_type, ia_name)

IAMark

Table Name: ia_assessment_iamark

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
ia_test	FK → IATestSetup	Reference to test	Required
student	FK → Student	Student record	Required
obtained_marks	Decimal(5,2)	Marks scored	Default: 0

Unique Constraint(ia_test, student)

IAAggregate

Table Name: ia_assessment_iaaggregate

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
student	FK → Student	Student reference	Required
subject	FK → Subject	Subject reference	Required
total_marks	Decimal(6,2)	Sum across IA tests	Default: 0
average_marks	Decimal(5,2)	Average across IA tests	Default: 0

Unique Constraint

(student, subject)

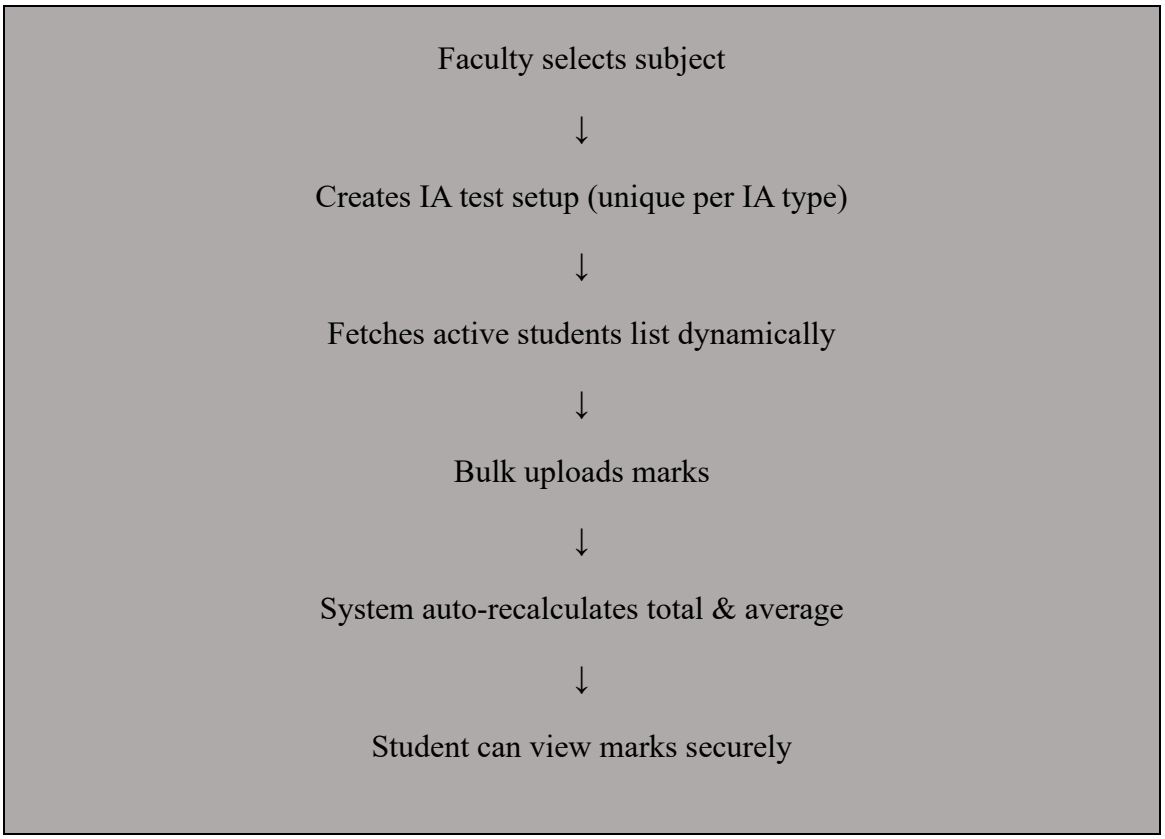
4.7.5 API Endpoints

Method	URL	Purpose	Permissions
POST	/api/ia-tests/create/	Create IA test setup	Authenticated Faculty
GET	/api/faculty/active-students/	Fetch valid students for IA entry	Authenticated Faculty
PUT	/api/ia-tests//update-marks/	Bulk create/update marks	Authenticated Faculty
GET	/api/faculty/ia-marks/	Faculty view marks per subject	Authenticated Faculty
GET	/api/student/ia-marks/	Student view personal IA marks	Authenticated Student

4.7.6 Permissions & Access Rules

Role	Access
Faculty	Create IA tests, enter/update marks, view IA
Student	View own marks only
HOD/Principal/Admin	Can add read/list endpoints if needed

4.7.7 Internal Workflow



4.7.8 Business Rules

- A faculty cannot create duplicate IA types for same subject + name.
- Students considered only when their status = "Active".
- Maximum marks cannot be changed after marks submission (recommended future rule).
- Obtained marks cannot exceed max_marks.

4.7.9 Validations

- Subject must belong to same department-semester-year as student.
- Invalid or missing subject_id returns meaningful errors.
- Numeric checks required for marks.
- IA must belong to currently logged-in faculty.

4.7.10 UI Screens (Frontend)

- IA Test Setup Form
- Active Students List (Auto-filtered)
- Bulk Mark Entry Page (Grid/Table)
- Faculty IA Reports
- Student IA Performance Dashboard

4.7.11 Error Handling & Status Codes

Status	Meaning
200	Success
201	IA setup created
400	Invalid input
401	Not authenticated
403	Permission denied
404	No matching data
409	Duplicate IA test
500	Server error

4.8 Sub-App Name: HOD Assignment & Role Transition Management App

4.8.1 Purpose & Responsibility

This sub-application manages **Head of Department (HOD) assignments**, including tracking active & previous HODs, updating faculty roles, ensuring only one active HOD per department, and synchronizing role changes with the authentication module (CustomUser).

It ensures **accurate departmental leadership assignment**, **role consistency**, and **automated archival of previous HODs**.

4.8.2 Key Features

- Assign a faculty member as HOD for a specific department.
- Automatically terminate previously assigned HOD (versioning of assignments).
- Maintain historical records of all previous HODs.
- Update role changes in both Faculty and CustomUser models.
- Fetch current and previous HOD lists using separate APIs.
- Role security — only Admin can assign HOD.

4.8.3 Folder / File Structure

```
backend/
└── hod_assignment/
    ├── apps.py
    ├── admin.py
    ├── test.py
    ├── models.py
    ├── serializers.py
    ├── views.py
    ├── urls.py
```

4.8.4 Database Model

Model: HODAssignment

Table Name: hod_assignment_hodassignment

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
faculty	FK → Faculty	Assigned HOD	Required
department	FK → Department	Assigned department	Required
assigned_date	Date	Date on which assignment starts	Auto
end_date	Date	Date of relieving	Nullable

Custom Model Method

- is_current() → returns True if end_date is NULL

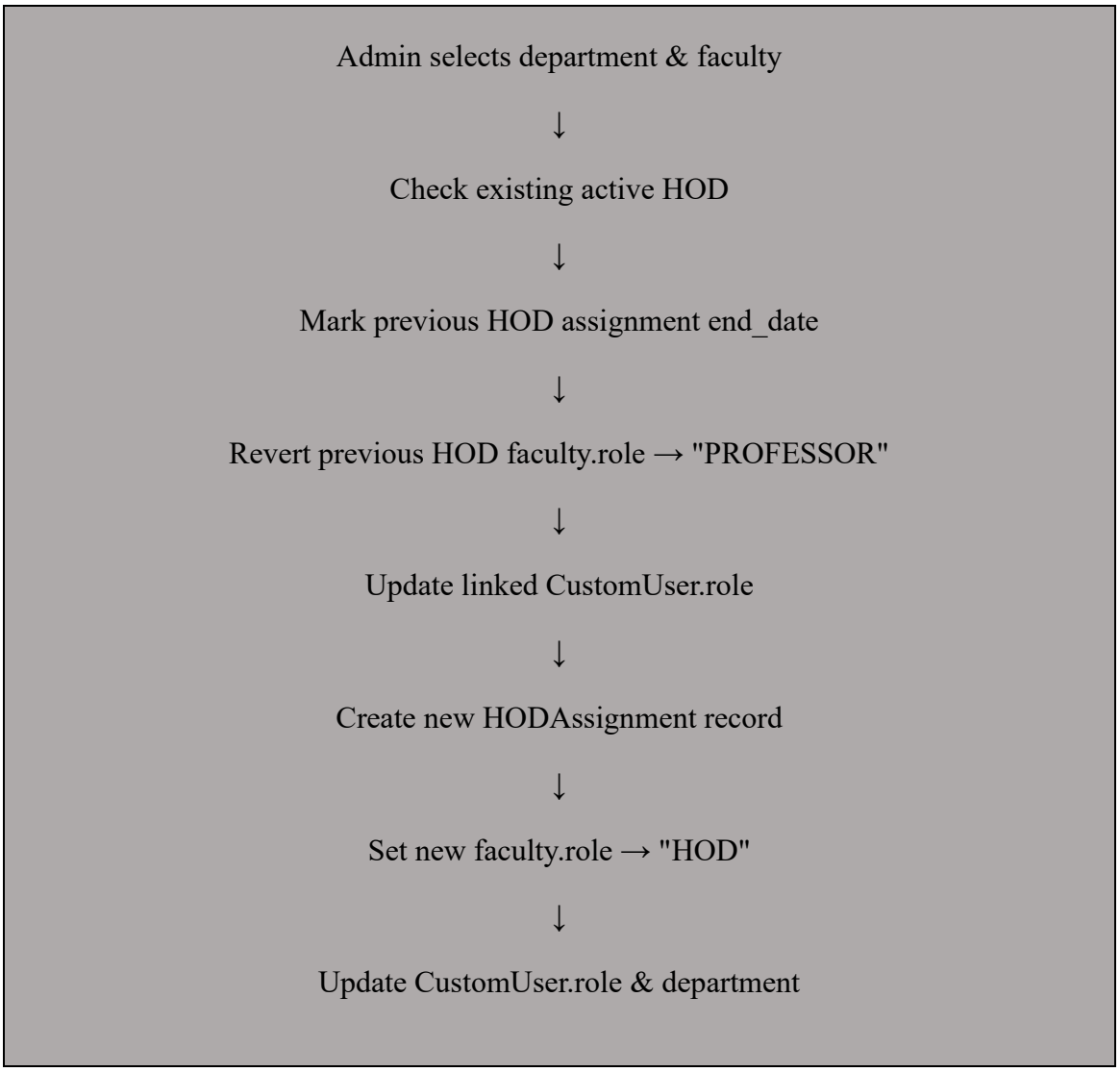
Model Behavior Rules

- Only **one active HOD** per department at any given time.
- When assigning new HOD:
    - All existing active HOD records are auto-closed via end_date.
    - Previous HOD's Faculty.role gets reverted to default.
    - New HOD's role updated to "HOD".
    - Related CustomUser model gets updated.

4.8.5 API Endpoints

Method	URL	Purpose	Permissions
GET	/api/hod/current/	Get all current HODs	Public
GET	/api/hod/previous/	Get past HOD history	Public
POST	/api/hod/assign/	Assign new HOD / Replace previous HOD	Admin Only

4.8.6 Assignment Workflow Logic



4.8.7 Permissions & Access Rules

Role	Privilege
Admin	Can assign/change HOD
Public	Can view current & previous HOD list
Faculty	Cannot assign themselves or others
Principal / HOD	Optional upgrade for future validation

4.8.8 Business Rules

- Only **one current HOD** per department → enforced via end_date=NULL.
- Faculty must belong to academic system but **may belong to any department prior to assignment** — department is auto-updated.
- end_date determines **historical vs active** status.

4.8.9 Validations & Edge Cases

- Prevent assigning same faculty repeatedly without closure.
- Ensure faculty and department fields are not null.
- CustomUser update is handled safely using try/except to avoid app crashes.
- Transactional operations guarantee **atomicity**.

4.8.10 UI / Frontend Screens

- HOD Assignment Panel (Admin only)
- Current HOD Table (Public/Admin/HOD/Principal)
- Past HOD Overview (Audit & HR use case)

4.8.11 Error Handling

HTTP Code	Description
200	Success
201	Assignment Created
400	Invalid data / Missing fields
401	Unauthorized
403	Admin permission required
409	Faculty already assigned
500	Internal server error

4.9 Sub-App Name: Attendance Management & Reporting System

4.9.1 Purpose & Responsibility

This sub-application handles **daily class attendance capturing**, **session-based attendance tracking**, and **attendance reporting** for students per subject. It allows faculty to record attendance per class session and provides summarized and detailed attendance insights for both **faculty and students**.

4.9.2 Key Features

- Create class-wise attendance sessions.
- Mark attendance per student for each session.
- Supports **multiple sessions per day** for same subject.
- Provides **subject-wise attendance reports** including percentage.
- Students can view **individual detailed attendance logs**.
- Faculty can fetch only **active, assigned students**.
- Flexible and scalable structure usable for **theory or lab** classes.

4.9.3 Folder / File Structure

```
backend/
└── attendance/
    ├── apps.py
    ├── admin.py
    ├── test.py
    ├── models.py
    ├── serializers.py
    ├── views.py
    ├── urls.py
```

4.9.4 Database Models

Model: AttendanceSession

Table Name: attendance_attendancesession

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
subject	FK → Subject	Subject for which attendance is taken	Required
faculty	FK → Faculty	Faculty taking the attendance	Required
date	Date	Session date	Required
session_count	Integer	Number of class sessions (hours)	Default = 1

Note: session_count allows tracking **double/triple hour classes**.

Model: AttendanceRecord

Table Name: attendance_attendancerecord

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
session	FK → AttendanceSession	Parent session	CASCADE
student	FK → Student	Student attendance record	Required
status	CharField	Present / Absent	Choices: (“Present”, “Absent”)

4.9.5 API Endpoints

Method	URL	Purpose	Permissions
POST	/api/attendance/session/add/	Create attendance session	Authenticated Faculty
POST	/api/attendance/record/add/	Insert per-student attendance	Authenticated Faculty

GET	/api/attendance/report/?subject_id=ID	Full subject-wise report	Authenticated
GET	/api/attendance/student/{id}/	Individual attendance history	Authenticated
GET	/api/attendance/students/?subject_id=ID	Fetch students eligible for attendance	Authenticated

4.9.6 Attendance Workflow

Faculty selects subject

↓

System fetches only active & assigned students

↓

Create attendance session (date + hour count)

↓

Record attendance for each student

↓

Generate per-subject & per-student reports

4.9.7 Permissions & Access Control

Role	Privilege
Faculty	Can create sessions & records
Student	Can view only their own attendance
Admin	Can access all data
Others	No access

4.9.8 Business Rules

- Each session must be linked to correct subject & faculty.
- Students are fetched based on **subject assignment scope**.
- Students must be **active** to appear in attendance list.
- No duplicate attendance entry allowed per (session + student).

4.9.9 Validations

- session_count >= 1
- Must send valid subject_id when fetching students.
- Subject must exist before session creation.
- Attendance percentage must avoid division by zero.

4.9.10 Error Handling

Status	Message
400	Missing required params or invalid subject id
401	Not authenticated
404	Student or session not found
409	Duplicate attendance record (future enhancement)
500	System or server-side failure

4.10 Sub-App Name: External Examination Results Upload, Validation & Audit System

4.10.1 Purpose & Responsibility

This sub-application enables students to **securely upload external exam result PDFs**, while the system automatically **validates authenticity** using USN & semester detection.
HOD/Admin can **view, manage, verify, update, or delete uploaded results**, and every modification is **logged for audit compliance**.

4.10.2 Key Features

- Students upload verified external result PDFs.
- Automatic validation based on:
    - USN text match in PDF
    - Semester consistency (digit & Roman numeral supported)
- Duplicate prevention per **student + semester + exam type**
- HOD/Admin can approve, update, or delete records.
- Auto creation of **audit logs for modifications**.
- Secure role-based permissions (Owner + HOD only).

4.10.3 Folder / File Structure

```
backend/
└── external_results/
    ├── apps.py
    ├── admin.py
    ├── test.py
    ├── models.py
```

```
├── serializers.py
├── permissions.py
├── views.py
├── urls.py
```

4.10.4 Database Models

Model: ExternalResult

Table Name: external_results_externalresult

Field	Type	Description	Constraints
id	AutoField	Primary Key	PK
student	FK → User	Owner / student uploading	Required
semester	FK → Semester	Result semester	Required
exam_type	CharField	FRESH / RV / MAKEUP	Required
pdf_file	FileField	Uploaded PDF	Required
uploaded_at	DateTime	Upload timestamp	Auto

Unique Constraint

(student, semester, exam_type)

Model: ChangeLog

Table Name: external_results_changelog

Field	Type	Description
id	AutoField	Primary Key
result	FK → ExternalResult (nullable)	Affected result
student	FK → User	Result owner
action_by	FK → User	Staff performing action
action	CharField	updated / deleted
timestamp	DateTime	Log creation

notes	Text	Details of modification

Used for **audit history + compliance tracking**.

4.10.5 Permissions & Access Matrix

Action	Student	HOD / Staff	Notes
Upload Result	✓	✗	Students only
View Own Result	✓	✓	Staff can view all
View All Results	✗	✓	Staff only
Update / Replace	✗	✓	Staff only
Delete	✗	✓	Staff only

Permission class used: **IsOwnerOrHOD**

4.10.6 PDF Validation Rules

Upon upload, system checks:

✓ Valid PDF document

✓ Extractable text using **PyMuPDF (fitz)**

✓ Contains student USN (case-insensitive, whitespace removed, hyphen tolerant)

✓ Semester inside PDF must match via either:

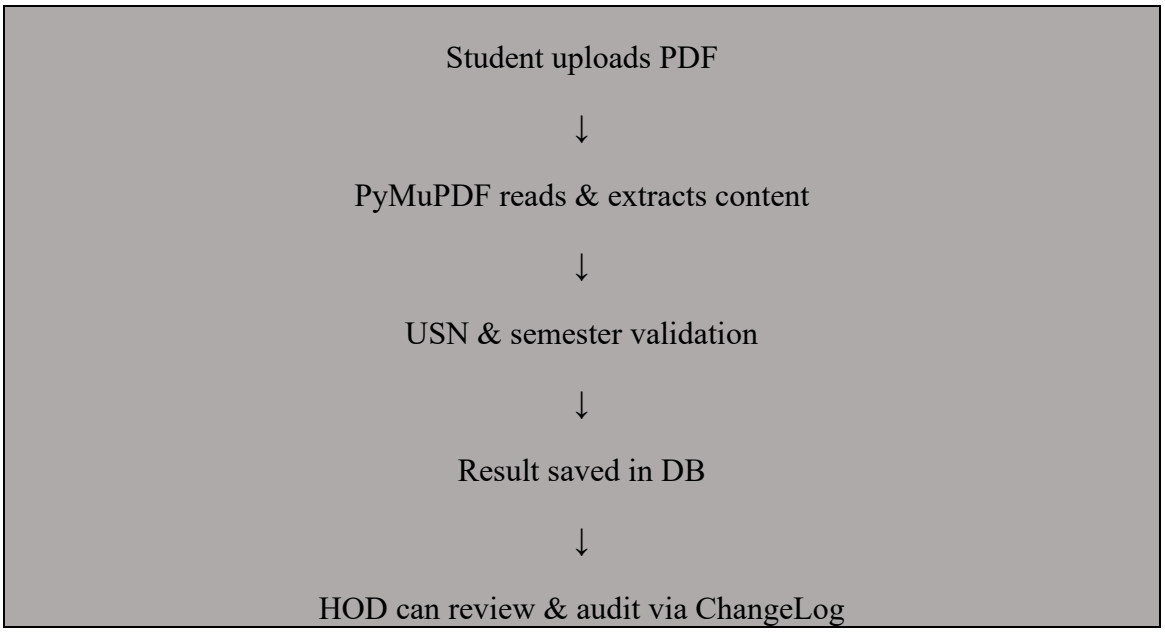
- Number (e.g., "Semester: 5")
- Pattern ("5th Semester", "SEMESTER5")
- Roman ("V Semester")

If any check fails → **400 Validation Error**

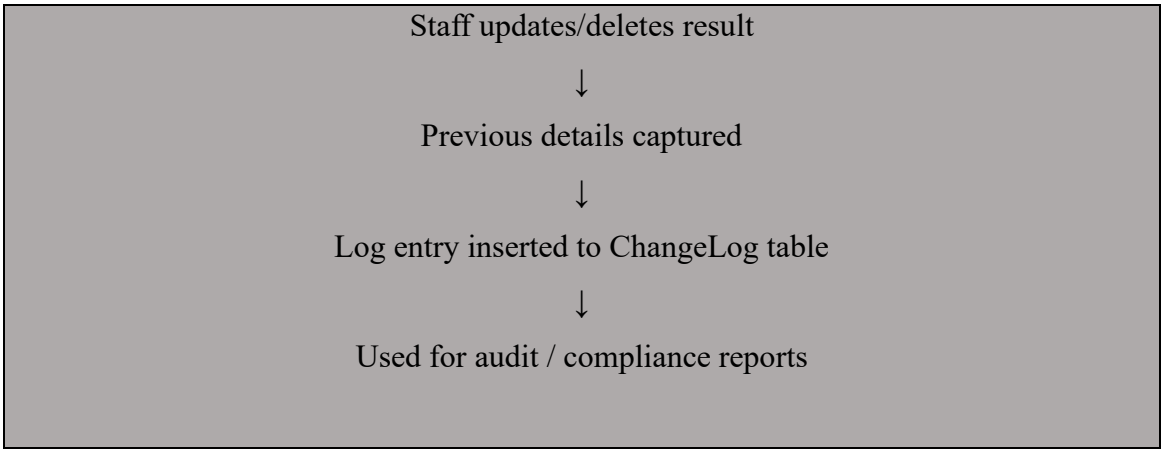
4.10.7 API Endpoints

Method	URL	Purpose	Permissions
GET	/api/external-results/	List results	Student: own, HOD: all
POST	/api/external-results/	Upload result	Student only
GET	/api/external-results/{id}/	Fetch single record	Owner / Staff
PUT/PATCH	/api/external-results/{id}/	Update PDF	Staff only
DELETE	/api/external-results/{id}/	Remove record	Staff only

4.10.8 Workflow (Upload → Validate → Access)



4.10.9 ChangeLog Workflow



4.10.10 Error Handling & Status Codes

Status	Description
200	Success
201	Created
400	Invalid PDF / semester mismatch
401	Authentication required
403	Permission denied
404	Result not found
409	Duplicate record
500	Unexpected system error

4.10.11 Business Rules

- One result per **(student, semester, exam_type)** to prevent duplication.
- USN & semester must match **exact student profile**.
- File pointer reset ensures secure re-save.
- Students cannot modify or delete previous uploads.
- File cleanup relies on storage settings (optional enhancement).

4.11 Sub-App Name: Fees & Payment Management System

4.11.1 Purpose & Responsibility

This sub-application manages **fee allocation, payment tracking, verification, receipts, and approval workflow** based on student seat category, academic year, and department structure.

It ensures:

- Accurate fee structuring by Accounts Department.
- Automatic mapping of approved fee structures to eligible students.
- Tracking of partial & full payments.
- Verification workflow and receipt issuance.
- Secure digital evidence upload for UPI or Bank transactions.

4.11.2 Key Features

- Create/update fee structures with multiple components.
- Principal approval of fee structure.
- Auto-assign fees to eligible students.
- Individual fee wallet: total, paid, balance.
- Support for **UPI & bank transfer** payments with evidence.
- Verification workflow for Accounts team.
- Auto receipt generation using unique receipt number.
- Optional notification trigger after verification.

4.11.3 Folder / File Structure

```
backend/
└── fees/
    ├── apps.py
    ├── admin.py
    ├── test.py
    ├── models.py
    ├── serializers.py
    ├── permissions.py
    ├── views.py
    ├── urls.py
```

4.11.4 Database Models

Model: FeeStructure

Table Name: fees_feestructure

Field	Type	Description
id	PK	Primary key
department	FK → Department	Applicable department
academic_year	FK → AcademicYear	Year of admission
semester	FK → Semester	Applicable semester
seat_type	Enum(GM,SNQ,Lateral)	Fee category
title	CharField	Structure name
description	Text	Optional
is_active	Boolean	Allowed to assign
approval_status	Enum	Pending/Approved/Rejected
approved_by	FK → User	Principal
approved_on	DateTime	Approval date

Unique Combination

(department, academic_year, semester, seat_type, title)

Model: FeeComponent

Table: fees_feecomponent

Field	Type	Description
fee_structure	FK	Parent fee
name	CharField	e.g. Tuition, Exam, Hostel
amount	Decimal	Amount

Model: StudentFee

Table: fees_studentfee

Field	Type	Description
student	FK → Student	Fee owner
fee_structure	FK	Linked structure
due_date	Date	Optional
status	Enum(Pending/Partial/Paid)	Fee status

Model: FeePayment

Table: fees_feepayment

Field	Type	Description
student_fee	FK	Student fee wallet
amount	Decimal	Paid amount
mode	Enum(UPI,BANK)	Payment type
upi_txn_id	Char	UPI reference ID
upi_screenshot	File	Mandatory for UPI
bank_statement_file	File	Mandatory for bank
verified	Boolean	Approved by accounts

entered_by	FK	User creating record
remark	Text	Optional

Model: FeeReceipt

Table: fees_feereceipt

Field	Type	Description
payment	OneToOne → FeePayment	Verified payment
receipt_no	Char	System generated
verified_by	FK → User	Accounts staff

4.11.5 API Endpoints

Method	URL	Purpose	Role
GET	/api/fees/fee-structures/	List fee structures	All
POST	/api/fees/fee-structures/	Create structure	Accounts
POST	/api/fees/fee-structures/{id}/approve/	Approve structure	Principal
GET	/api/fees/student-fees/	All student fees	Admin/Accounts
POST	/api/fees/student-fees/bulk-assign/	Auto map students	Accounts
GET	/api/fees/student-fees/my-fees/	Student dashboard	Student
POST	/api/fees/payments/	Upload payment	Student
POST	/api/fees/payments/{id}/verify/	Verify payment	Accounts
GET	/api/fees/receipts/	List receipts	Student/Admin

4.11.6 Permission & Roles

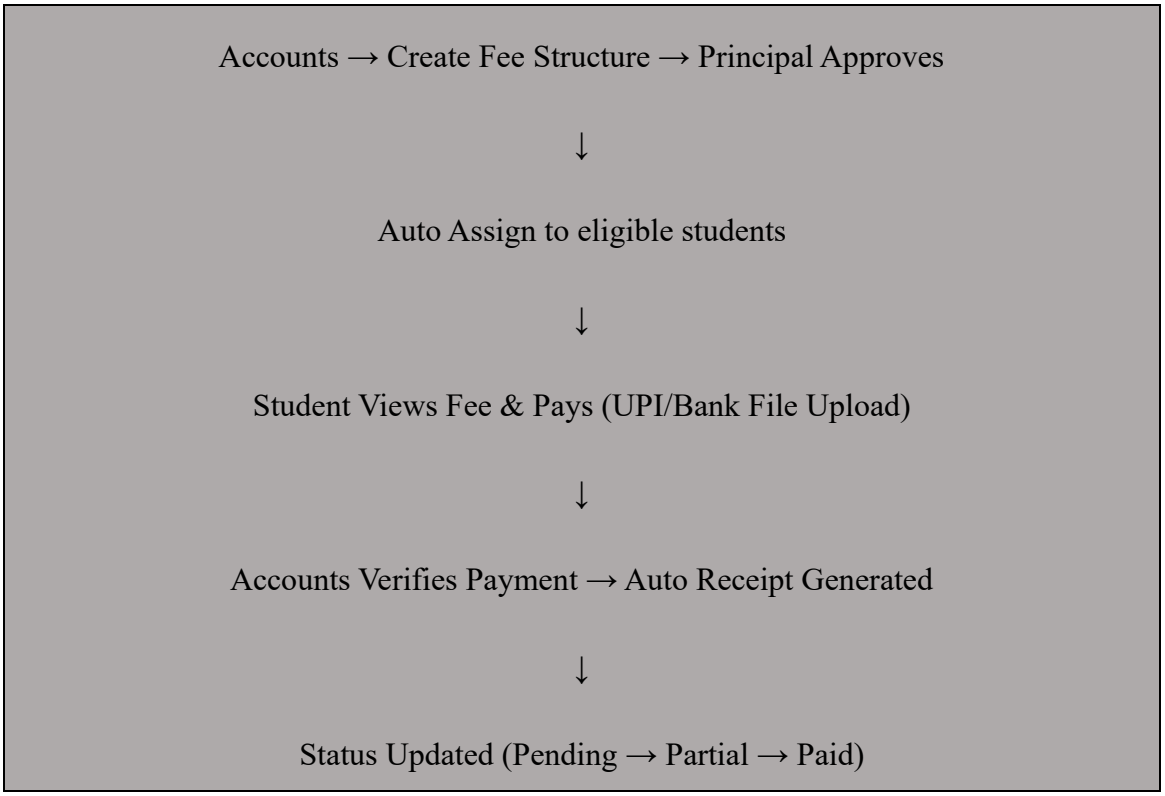
Feature	Student	Accounts	Principal	Admin
View fee structure	✓	✓	✓	✓
Create structure	✗	✓	✗	✓
Approve structure	✗	✗	✓	✓

Upload payment	✓	✕	✕	✕
Verify payment	✕	✓	✕	✓
Get receipts	✓	✓	✓	✓

4.11.7 Validation Rules

- A student **can have only one StudentFee per structure**.
- Payment verification allowed **only if uploaded proof matches mode**.
- Status update logic:
- Paid if balance_amount == 0
- Partial if balance > 0
- Fee assignment only allowed if structure is **approved**.

4.11.8 Flow Diagram (High Level)



4.11.9 Data Flow (CRUD)

Entity	C	R	U	D
Fee Structure	✓	✓	✓	✓
Fee Component	✓	✓	✓	✓
Student Fee	Auto & Manual	✓	✗	✗
Fee Payment	✓	✓	✗	✗
Receipt	Auto	✓	✗	✗

4.11.10 Business Rules

- No modification allowed after fee payment starts.
- Only verified payments count toward **paid_amount**.
- Receipt generated **only once** per payment.
- Screenshot & reference mandatory for security proof.
- Wrong seat type selection results in **zero assignment**.

4.11.11 Error Handling & Status Codes

Status	Reason
400	Missing mandatory fields
403	Role permission denied
404	Student/Structure not found
409	Duplicate fee mapping
422	Invalid payment mode proof
500	Unexpected transaction error

4.12 Sub-App Name: Grievance Management System

4.12.1 Purpose & Responsibility

This module enables students to officially raise complaints, issues, or requests and ensures proper routing, accountability, and resolution by the appropriate authority (Faculty / HOD / Principal / Accounts / System Admin).

It provides:

- Formal grievance submission
- Automated role-based assignment
- Status tracking
- Resolution recording and accountability

4.12.2 Key Features

- Student-only grievance submission
- Role-based targeting (Faculty / HOD / Principal / Accounts / Exam / Library / Scholarship etc.)
- Auto-assignment of responsible authority
- Controlled visibility based on role
- Restricted update + response actions
- Status tracking: Pending → In-Progress → Resolved

4.12.3 Folder & File Structure

```
backend/
└── grievances/
    ├── apps.py
    ├── admin.py
```

```
├── test.py
├── models.py
├── serializers.py
├── permissions.py
├── views.py
├── urls.py
```

4.12.4 Models

Model: Grievance

Table Name: grievances_grievance

Field	Type	Description
id	PK	Autogenerated
student	FK → Student	Complaint raised by
topic	CharField	Short title
description	Text	Full grievance details
target_role	CharField (ROLE_CHOICES)	Role intended to resolve
target_user	FK → CustomUser	Assigned authority
status	Enum(Pending, In Progress, Resolved)	Current state
response	Text	Final reply or action notes
created_at	DateTime	Auto timestamp
updated_at	DateTime	Auto timestamp

Relationships

Relation	Type
Student → Grievance	One-to-Many
User (Role-based) → Assigned grievances	One-to-Many

4.12.5 API Endpoints

Method	URL	Description	Access
GET	/api/grievances/	List grievances	Student / Assigned users
POST	/api/grievances/	Submit grievance	Student
GET	/api/grievances/{id}/	View detail	Student / Assigned user
PUT/PATCH	/api/grievances/{id}/	Update (restricted)	Assigned or Admin
DELETE	/api/grievances/{id}/	Delete	Admin only
POST	/api/grievances/{id}/respond/	Respond & mark resolved	Assigned User Only

4.12.6 Permissions / Role Rules

Action	Student	Target User	Admin
Create grievance	✓	✗	✗
List grievances	✓ own	✓ assigned	✓
Respond	✗	✓	✓
Update (status/response)	✗	✓	✓
Delete	✗	✗	✓

4.12.7 Flow Diagram

Student → Submit Grievance

↓ (auto detect role + department)

System → Assign target_user

↓

Target User → View & Respond

↓

Grievance → Mark as Resolved

↓

Recorded for future audit

4.12.8 Business Rules

- Only **students** can raise grievances.
- The **target_role** is mandatory.
- System automatically maps user based on:
    - Role
    - Active status
    - Department (if HOD/Accounts/Principal)
- Once resolved, grievance becomes **read-only except admin**.
- Only **assigned authority** can respond.

4.12.9 Validations

- Topic must not be empty.
- Description must be minimum meaningful text.
- Target role must belong to valid ROLE_CHOICES.
- Target user must exist and be active; else raise error.
- Duplicate open grievances on same topic can optionally be prevented.

4.12.10 Error Handling

Code	Cause
400	Missing required fields
403	User not allowed to perform action
404	No matching authority found
409	Target duplicate grievance logic (optional)

4.13 Sub-App Name: Library Management System (LMS)

4.13.1 Purpose & Responsibility

To manage all library-related operations including physical books, e-books, book copies, issuing & returning process, fine calculation, audit log tracking, and special purchase requests raised by students or faculty.

This module ensures:

- Proper tracking & issuing of library materials
- Accountability through audit logs
- Role-based permissions (Library Officer, Principal)

4.13.2 Key Features

- Add & Manage Books
- Auto-barcode generation for each book copy
- Maintain issued / available inventory
- Issue & Return workflow
- Lost-book & Fine handling
- E-Book upload & access
- Book purchase request & approval flow
- Audit logs for every critical operation

4.13.3 Folder / File Structure

```
backend/
└── library/
    ├── apps.py
    ├── admin.py
    ├── test.py
    ├── models.p
```

```
├── serializers.py
├── views.py
├── signals.py
├── permissions.py
├── urls.py
```

4.13.4 Database Models

Book

Field	Type	Description
id	PK	Primary key
title	CharField	Main title
subtitle	CharField	Optional
authors	CharField	Multiple authors
edition	CharField	Edition no.
publisher	CharField	Publishing house
department (FK)	Department	Linked department
category	CharField	Genre/Type
language	CharField	Book language
isbn	CharField	ISBN code
mrp	Decimal	Price
description	Text	Summary
created_at	DateTime	Added timestamp
updated_at	DateTime	Last updated

Computed Properties

- total_copies
- available_copies
- issued_copies
- is_out_of_stock

BookCopy

Field	Type
id	PK
book (FK)	Book
copy_number	Integer
barcode	Auto-generated
shelf_location	Auto-filled
status	Enum(available / issued / lost / damaged / maintenance)
created_at	DateTime

Business Rule:

unique_together = (book, copy_number)

EBook

Field	Type
id	PK
book (OneToOne)	Book
file	FileField
external_link	URL
uploaded_by	User
uploaded_at	DateTime
downloads, views	Counters

BookRequest

Field	Type
requester	User
title, authors	String
reason	Text
urgency	String

status	Enum → pending / verified / approved / purchased / rejected
forwarded_to	User
timestamps	created_at / updated_at

IssueRecord

Field	**Type**
issue_id	UUID
borrower	User
copy (FK)	BookCopy
issued_at	DateTime
due_date	Date
returned_at	DateTime
status	issued / returned / lost
created_by	User

Fine

Field	**Type**
issue (FK)	IssueRecord
amount	Decimal
reason	CharField
assigned_by	User
assigned_at	DateTime
paid	Boolean
paid_at	DateTime
payment_reference	String

AuditLog

Field	Type
actor	User
action	CharField
model_name	CharField
object_id	String
timestamp	DateTime
extra	JSON

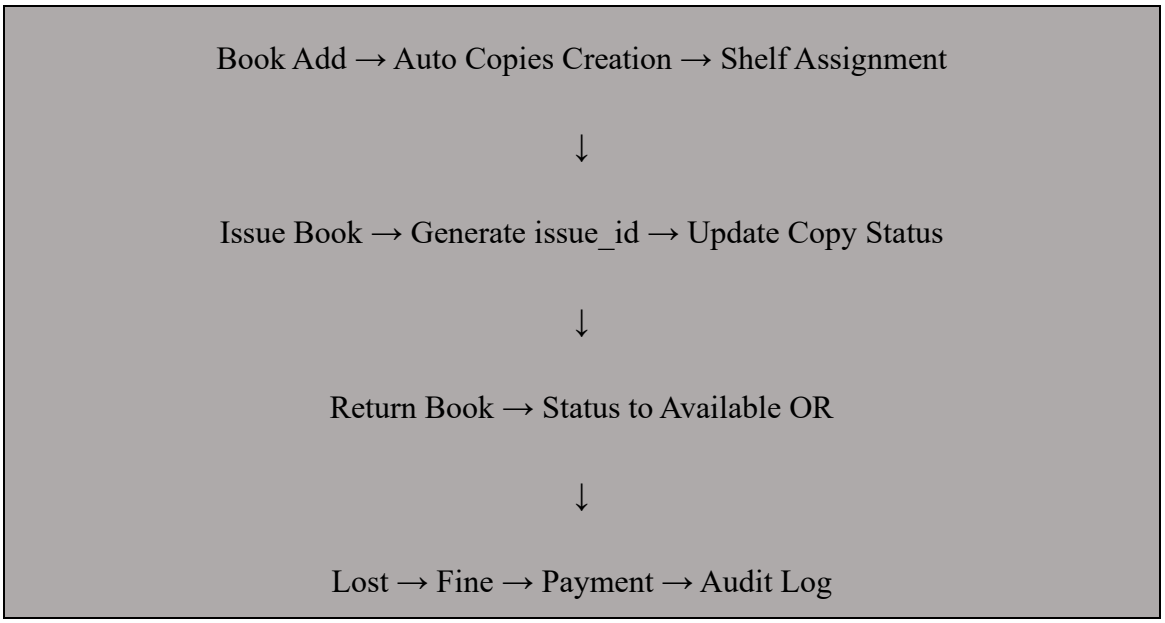
4.13.5 API Endpoints

Method	Endpoint	Description	Role
GET/POST	/library/books/	List or Create Books	Library Officer
POST	/library/books/{id}/add_copies/	Add copies	Library Officer
GET/POST	/library/copies/	Manage book copies	Library Officer
GET/POST	/library/ebooks/	Upload/View ebooks	Faculty/Officer
GET/POST	/library/requests/	Book purchase requests	All users
POST	/library/requests/{id}/verify/	Verify request	Library Officer
POST	/library/requests/{id}/final_approve/	Principal approval	Principal
POST	/library/issues/	Issue a book	Library Officer
POST	/library/issues/{id}/return_book/	Return book	Library Officer
POST	/library/issues/{id}/mark_lost/	Mark lost & fine	Library Officer
POST	/library/fines/{id}/mark_paid/	Mark fine paid	Borrower

4.13.6 Permission Matrix

Action	Student	Faculty	Library Officer	Principal	Admin
View Books	✓	✓	✓	✓	✓
Add Books	✗	✗	✓	✗	✓
Issue Books	✗	✗	✓	✗	✓
Upload E-Book	✗	✓	✓	✓	✓
Approve Requests	✗	✗	✗	✓	✓
Mark Lost	✗	✗	✓	✗	✓

4.13.7 Flow Diagram



4.13.8 Business Rules

- Cannot issue if copy status != available
- Auto-generate barcode format → TITLE-###
- Fine auto-created if book lost
- Request must be verified before principal approval
- E-book must map to existing physical book

4.13.9 Error Handling & Status Codes

Code	Scenario
400	Copy not available
401	Authentication required
403	Not authorized
404	Book/Copy/IRequest not found
409	Duplicate copy barcode

4.14 SDMS Admin Panel Configuration Documentation

The **Admin Panel** in the **Student Data Management System (SDMS)** is implemented using Django's built-in **Admin Framework**.
The primary purpose of this configuration is to:

- Manage records using a secure backend
- Provide CRUD operations without writing extra UI
- Offer filtered, searchable, and well-organized dashboards
- Enable institution authorities to manage data efficiently

The admin.py files inside each Django app define how models appear and behave in the admin console.

4.14.1 Custom User & Authentication Admin

Admin: CustomUserAdmin, PasswordResetOTPAdmin
Module: userlogin

Key Features:

- Custom user model integrated with roles (Student, Faculty, HOD, Principal, Accounts, Library Officer, Admin)
- Search and filter based on email, phone, department & role
- Permission flags like is_staff, is_superuser
- Supports OTP-based password reset monitoring

4.14.2 Timetable Management Admin

Admin: TimeTableAdmin, TimeTableSlotAdmin, TimeTableApprovalHistoryAdmin
Module: timetable

Key Features:

- Inline slot editing using TabularInline
- Versioning & approval visibility
- Filters based on department, semester, and academic year

4.14.3 Subject Management Admin

Admin: SubjectAdmin, SubjectAssignmentAdmin
Module: subjects

Key Features:

- Displays subject codes with department and semester
- Faculty assignment mapping table
- Searchable by faculty or subject code

4.14.4 Student Management Admin

Admin: StudentAdmin
Module: students

Key Features:

- Full student profile editable with grouped fieldsets
- Read-only generated IDs
- Filtering available by department, status, and gender

4.14.5 Library Management Admin

Admin: BookAdmin, BookCopyAdmin, EBookAdmin, BookRequestAdmin, IssueRecordAdmin, FineAdmin, AuditLogAdmin
Module: library

Key Features:

- Book inventory control with inline copy creation
- Issue, return, lost & fine tracking
- Auditing log for every operation

4.14.6 Internal Assessment Admin

Admin: IATestSetupAdmin, IAMarkAdmin, IAAggregateAdmin
Module: ia_assessment

Key Features:

- Manage IA tests and marking
- Display average marks & summaries

4.14.7 HOD Assignment Admin

Admin: HODAssignmentAdmin
Module: hod_assignment

Key Features:

- Track current and previous HOD history
- Editable end-date for past records

4.14.8 Grievance Management Admin

Admin: GrievanceAdmin
Module: grievances

Key Features:

- Monitor student complaints
- Role-based assigned staff view
- Status filtering for tracking

4.14.9 Fees & Accounting Admin

Admin: FeeStructureAdmin, FeeComponentAdmin, StudentFeeAdmin, FeePaymentAdmin, FeeReceiptAdmin
Module: fees

Key Features:

- Manage fee structure with inline components
- Track student fee status, receipts & payment verification
- Auto-calculated totals, balances, and paid amounts

4.14.10 External Results Upload Admin

Admin: ExternalResultAdmin, ChangeLogAdmin
Module: external_results

Key Features:

- Secure academic result upload
- Tracks changes with audit logs

4.14.11 Attendance Admin

Admin: AttendanceSessionAdmin, AttendanceRecordAdmin
Module: attendance

Key Features:

- View attendance sessions & detailed records
- Filters by faculty & subject

4.14.12 Academics & Master Data Admin

Admin: DepartmentAdmin, AcademicYearAdmin, SemesterAdmin, SchemaAdmin
Module: academics

Key Features:

- Maintains master data for all other modules

Summary Table

Module	Admin Configured	Purpose
userlogin	✓	Custom user & OTP system
students	✓	Student info management
faculty	✓	Faculty database
subjects	✓	Subject + assignment mapping
timetable	✓	Slot + version control
attendance	✓	Class attendance records
ia_assessment	✓	Internal tests & marks
fees	✓	Fee structure, payment, receipts
library	✓	Books, e-books, issuing
grievances	✓	Complaint redressal
external_results	✓	Result upload & verification
academics	✓	Master data setup
hod_assignment	✓	HOD allocation

4.15 SDMS Django Configuration Overview

This section explains the global configuration settings used in the **Student Data Management System (SDMS)** project implemented using **Django REST Framework**. The settings.py file defines the core backend configuration such as installed applications, middleware, authentication, database, email service, and security parameters, while the urls.py file maps all application-level API endpoints.

4.15.1 Project Information

The project was generated using:

django-admin startproject SDMS

Django Version: **4.2.x**
Programming Language: **Python 3.x**

The initial section includes Django documentation links and comments for development reference.

4.15.2 Path & Base Directory Setup

BASE_DIR = Path(__file__).resolve().parent.parent

- Defines the root directory of the Django project.
- Used for referencing files such as templates, static, and media files.

4.15.3 Security & Debug Settings

SECRET_KEY = "django-insecure-xxxxxxx..."
DEBUG = True
ALLOWED_HOSTS = []

- SECRET_KEY is used for encryption, password hashing, sessions, CSRF tokens, etc.
- DEBUG=True is only for development; must be set to False in production.
- ALLOWED_HOSTS remains empty for local development; must include actual domain/IP in deployment.

An optional data encryption key is also defined:
SALT_KEY = 'LzU8...'

4.15.4 Installed Applications

```
INSTALLED_APPS = [
    # Django default apps
    "django.contrib.admin",
    "django.contrib.auth",
    # Third-party apps
    "rest_framework",
    "rest_framework.authtoken",
    # Custom apps (Modules)
    "userlogin",
    "students",
    "faculty",
    "fees",
    "academics",
    "hod_assignment",
    "subjects",
    "timetable",
    "attendance",
    "ia_assessment",
    "library",
    "grievances",
    "external_results",
]
```

These represent **14 modular sub-applications**, enabling a **Micro-Modular Django Architecture**.

4.15.5 Middleware Registration

Middleware handles request/response processing and security layers.

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

4.15.6 Templates & Rendering

Templates directory registered for custom HTML rendering, especially for email templates or admin messages.

```
TEMPLATES = [{
    "DIRS": [BASE_DIR / "templates"],
}]
```

4.15.7 REST Framework Authentication

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
```

✓ Secures all APIs

✓ Requires token-based access

✓ Prevents unauthorized data exposure

4.15.8 Database Configuration

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'sdms_db',
        'USER': 'postgres',
        'PASSWORD': 'root',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

- PostgreSQL chosen for reliability, scalability, and JSON support.
- Credentials must be moved to .env for production.

4.15.9 Internationalization

```
LANGUAGE_CODE = "en-us"
TIME_ZONE = "UTC"
USE_I18N = True
USE_TZ = True
```

Supports multilingual and timezone-aware datetimes.

4.15.10 Static Files

```
STATIC_URL = "static/"
```

Used for frontend assets like CSS, JavaScript, and images.

4.15.11 Custom User Model

```
AUTH_USER_MODEL = 'userlogin.CustomUser'
```

Allows **role-based authentication** (Admin, Student, Faculty, HOD, Principal, Accounts, Library Officer, etc.).

4.15.12 Email Configuration (SMTP)

```
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_USE_TLS = True
EMAIL_PORT = 587
```

Used for login OTP, approval notifications, and alerts.

⚠ **Credentials must not be hard-coded** — use environment variables.

4.15.13 JWT Configuration

```
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=5),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
}
```

Enhances session security using token expiry.

4.15.14 URL Routing

All application routes are combined in urls.py:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('userlogin/', include('userlogin.urls')),
    ...
    path("external_results/", include("external_results.urls")),
]
```

✓ Clean modular structure

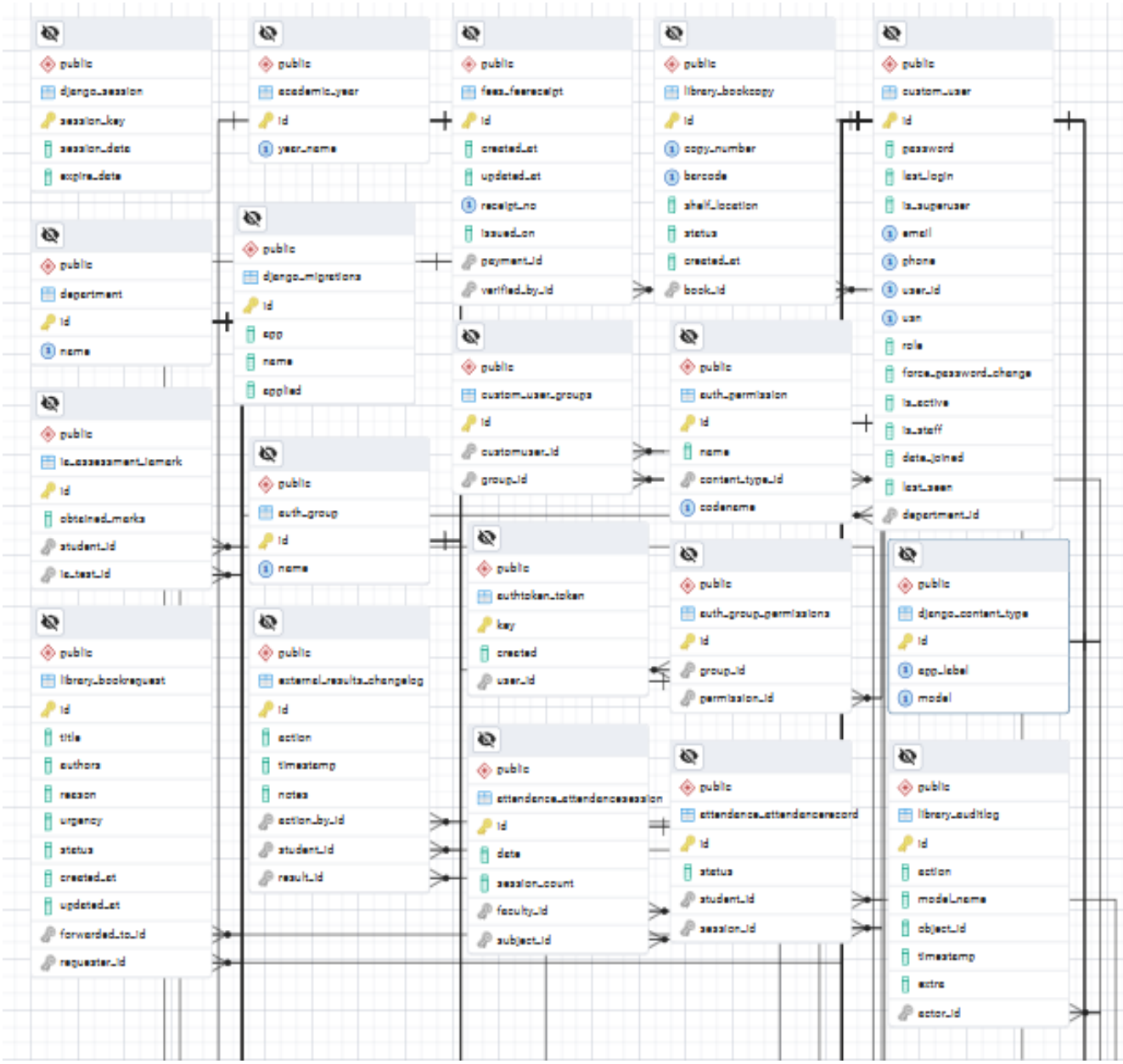
✓ Each app contains its own urls.py

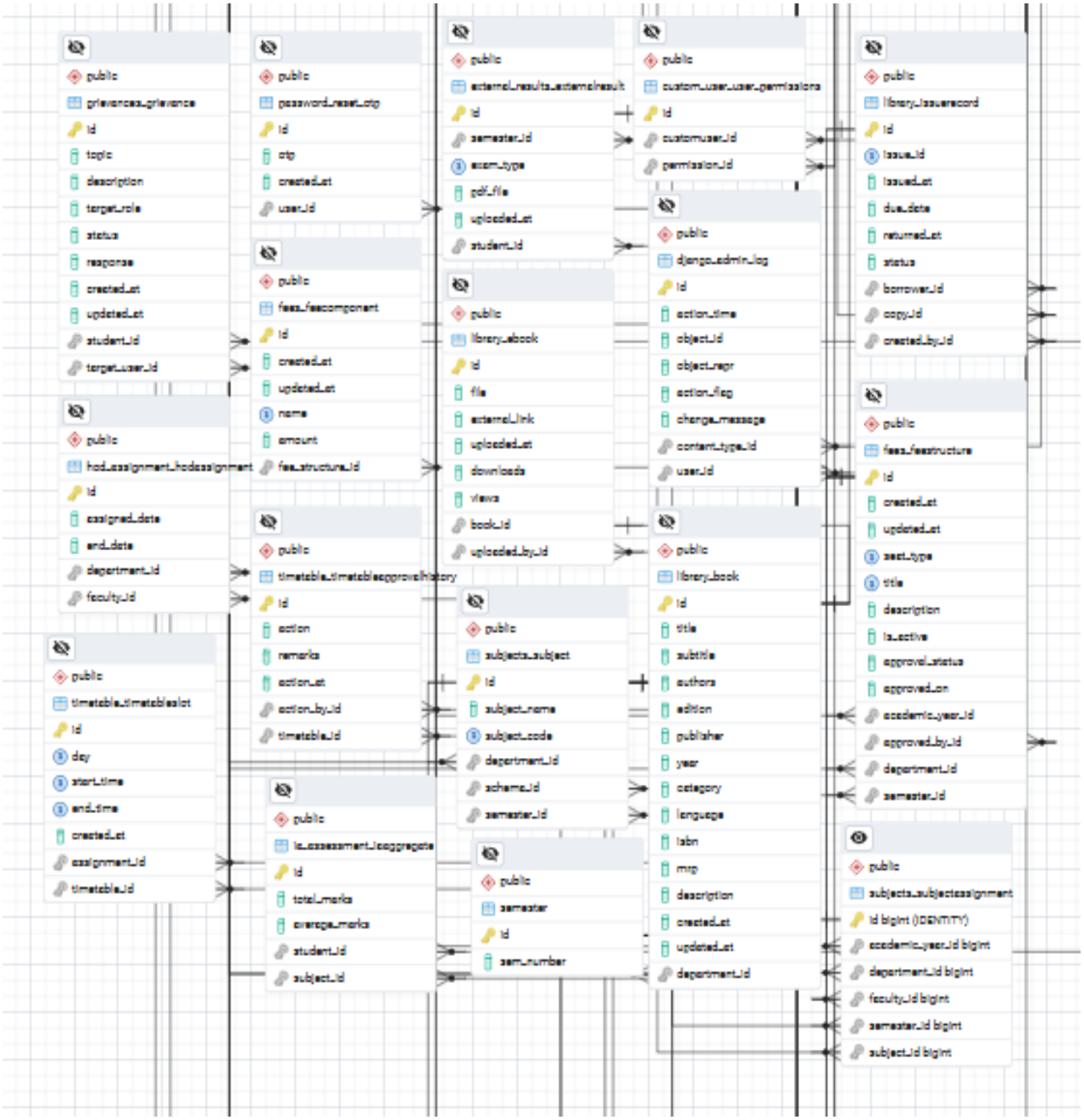
✓ Easier debugging + Extendability

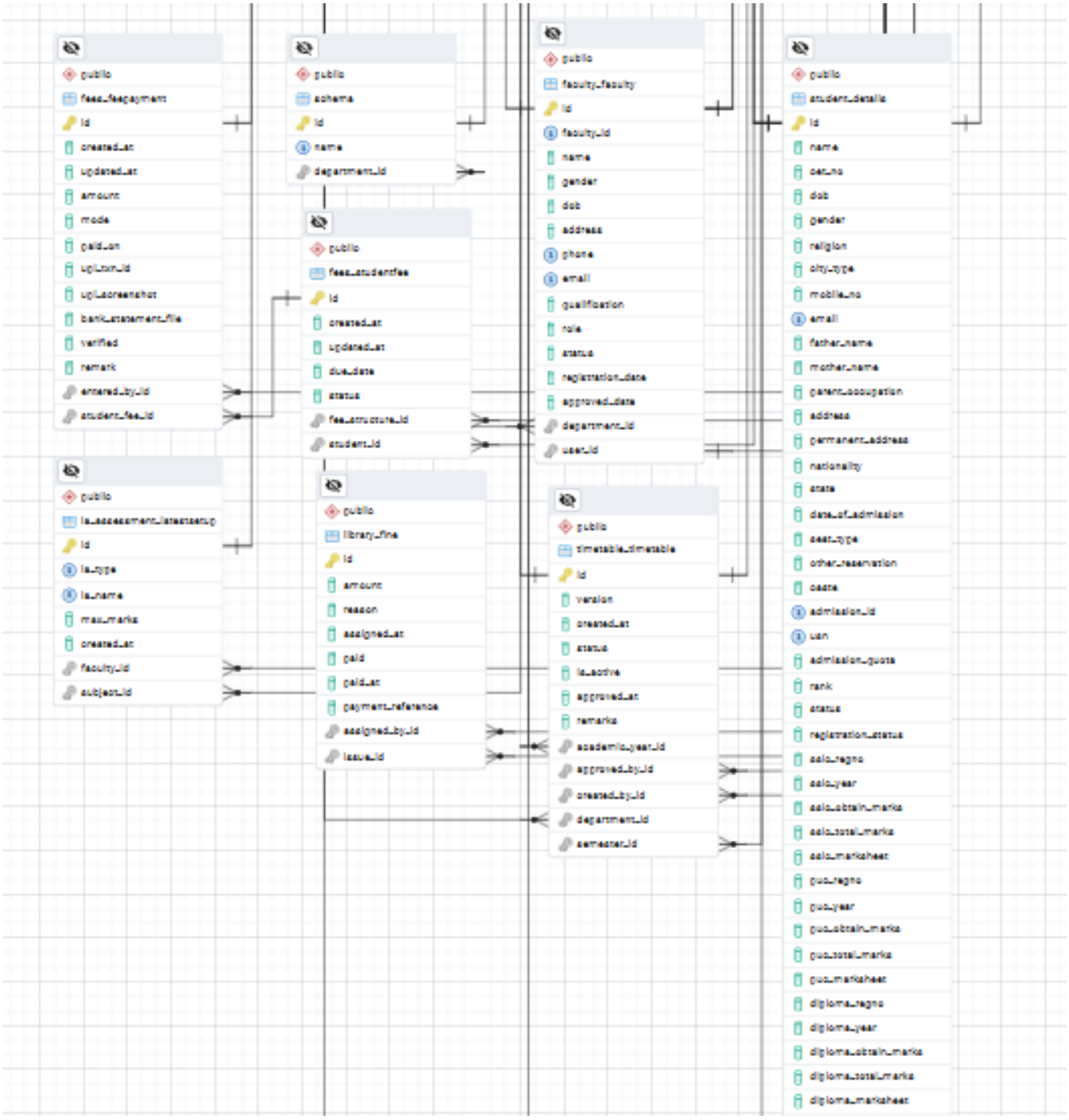
Static + media served during development:

```
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

5.Database E-R model




public
grievances_grievance
id
topic
description
target_role
status
response
created_at
updated_at
student_id
target_user_id
public
password_reset_otp
id
otp
created_at
user_id
public
fees_feecomponent
id
created_at
updated_at
name
amount
fee_structure_id
public
external_results_externalresult
id
semester_id
exam_type
pdf_file
uploaded_at
student_id
public
library_ebook
id
file
external_link
uploaded_at
downloads
views
book_id
uploaded_by_id
public
custom_user_user_permissions
id
customuser_id
permission_id
public
django_admin_log
id
action_time
object_id
object_repr
action_flag
change_message
content_type_id
user_id
public
library_issuerecord
id
issue_id
issued_at
due_date
returned_at
status
borrower_id
copy_id
created_by_id
public
hod_assignment_hodassignment
id
assigned_date
end_date
department_id
faculty_id
public
timetable_timetableapprovalhistory
id
action
remarks
action_at
action_by_id
timetable_id
public
library_book
id
title
subtitle
authors
edition
publisher
year
category
language
isbn
mrp
description
created_at
updated_at
department_id
public
fees_feestructure
id
created_at
updated_at
seat_type
title
description
is_active
approval_status
approved_on
academic_year_id
approved_by_id
department_id
semester_id
public
timetable_timetableslot
id
day
start_time
end_time
created_at
assignment_id
timetable_id
public
subjects_subject
id
subject_name
subject_code
department_id
scheme_id
semester_id
public
ia_assessment_iaaggregate
id
total_marks
average_marks
student_id
subject_id
public
semester
id
sem_number
public
subjects_subjectassignment
id bigint (IDENTITY)
academic_year_id bigint
department_id bigint
faculty_id bigint
semester_id bigint
subject_id bigint


public
fees_feepayment
id
created_at
updated_at
amount
mode
paid_on
upi_txn_id
upi_screenshot
bank_statement_file
verified
remark
entered_by_id
student_fee_id
public
schema
id
name
department_id
public
fees_studentfee
id
created_at
updated_at
due_date
status
fee_structure_id
student_id
public
la_assessment_lastassessg
id
la_type
la_name
max_marks
created_at
faculty_id
subject_id
public
library_fine
id
amount
reason
assigned_at
paid
paid_at
payment_reference
assigned_by_id
issue_id
public
faculty_faculty
id
faculty_id
name
gender
dob
address
phone
email
qualification
role
status
registration_date
approved_date
department_id
user_id
public
timetable_timetable
id
version
created_at
status
is_active
approved_at
remarks
academic_year_id
approved_by_id
created_by_id
department_id
semester_id
public
student_details
id
name
cet_no
dob
gender
religion
city_type
mobile_no
email
father_name
mother_name
parent_occupation
address
permanent_address
nationality
state
date_of_admission
seat_type
other_reservation
caste
admission_id
uin
admission_quota
rank
status
registration_status
sslc_regno
sslc_year
sslc_obtain_marks
sslc_total_marks
sslc_marksheet
puc_regno
puc_year
puc_obtain_marks
puc_total_marks
puc_marksheet
diploma_regno
diploma_year
diploma_obtain_marks
diploma_total_marks
diploma_marksheet

6.Security & Authentication

6.1 Authentication Method

Your system uses **Token-based Authentication** provided by **DRF + TokenAuthentication** with optional **JWT configuration**.

Feature	Implementation
Authentication Type	Token-based (DRF Token Auth) + JWT Ready
Token Storage	On server using DRF Token Model
Token Expiry	JWT settings configured (5 mins Access, 7 days Refresh)
Credential Login	Email + Password / OTP
Custom User Model	userlogin.CustomUser
Force Password Reset	force_password_change = True security flag

Why Token / JWT instead of Session Auth?

Token / JWT	Session
Best for mobile & API	Best for browser apps
Stateless & scalable	Stateful & server memory
Secure & short lived	Needs manual expiry

Therefore, SDMS uses **Token authentication** with JWT support for **distributed & modular API-based architecture**.

6.2 Role-Based Access Control (RBAC)

Roles are assigned at authentication level using **CustomUser.role**

Available Roles

- **ADMIN / SUPERADMIN**
- **STUDENT**
- **FACULTY**
- **HOD**
- **PRINCIPAL**
- **ACCOUNTS**
- **LIBRARY_OFFICER**

Role Access Matrix

Module	Student	Faculty	HOD	Principal	Accounts	Library Officer	Admin
Login & Profile	✓ View/Edit	✓ View/Edit	✓	✓	✓	✓	✓
Academics / Subjects	View	View	Manage	View	❌	❌	Full
HOD Assignment	❌	❌	View	Approve	❌	❌	Full
Timetable	View	View	Manage	Approve	❌	❌	Full
Attendance	View Own	Record	View	View	❌	❌	Full
IA Marks	View Own	Enter	View	View	❌	❌	Full
Fees	View Own	❌	View	View	Manage	❌	Full
Payments	Upload	❌	❌	❌	Verify	❌	Full
Library	View	View	View	View	View	Manage	Full
Grievances	Raise	Respond if assigned	Respond if assigned	Respond if assigned	Respond if assigned	❌	Full
External Results	Upload Own	❌	View	View	❌	❌	Full

6.3 Access Enforcement Locations

- Views & ViewSets
- DRF Permission Classes
- Serializer Validation
- API URL-based restrictions

Example used in modules:

```
class IsAccounts(IsRole):
    allowed_roles = ["ACCOUNTS"]
class IsOwnerOrHOD(BasePermission):
```

6.4 Input Validation & Sanitization Rules

Data Type	Validation Rule
Email	Must be unique / valid RFC format
Phone	Must be numeric + unique
Password	Strong constraints recommended
File Uploads	Limit type, size, extension
PDF (External Results)	Auto-USN verification
Amount	Decimal + non-negative
Text fields	Strip whitespace, no HTML, no scripts
Dates	Must be future/past validated as required

Prevention Steps

✓ backend serializer validation

✓ frontend form validation

✓ avoid string concatenation queries

✓ convert inputs to strict types

6.5 Sensitive Data Protection

Data Type	Protection
Passwords	**PBKDF2 hash** (default Django)
OTP Codes	**Stored in DB + auto-expiry**
Result PDFs	**Content validation + secure storage**
Tokens	**HTTP header only, never logged**
User IDs	**Not exposed publicly**
Fee Receipts & Financial records	**Read-only endpoints** after creation

Encryption & Security Configs

Feature	Status
HTTPS Recommended	✓
SECRET_KEY stored securely	✓ (Needs env var in production)
SALT_KEY added	✓
CSRF enabled	✓ for web
Debug disabled in production	To configure
File uploads restricted	✓ (validation in serializers)

6.6 Major Security Threat Mitigations

Threat	Mitigation Applied
SQL Injection	ORM + no raw queries
XSS	DRF + JSON only
CSRF	Middleware enabled
Brute Force	Token expiry + role check
Privilege Escalation	RBAC + strict view permissions
Identity Spoofing	File content check + unique constraints
Data Tampering	Logged change history (External Results / Fees)

6.7 Recommendations for Production

Category	Action
Environment	Store SECRET_KEY & DB credentials in .env
Rate Limiting	Add Django-throttling
Logging	Use audit trails everywhere
HTTPS	Configure SSL
Backup	Nightly DB backup schedule
Monitoring	Use Sentry + Pg_stat monitor

7.Full API Reference Guide

7.1. Authentication & User (Module: userlogin)

Endpoint	Method	Module	Request Example (JSON)	Sample Response	Status Codes	Roles Allowed
userlogin/login/	POST	userlogin	{ "identifier": "user@example.com", "password": "Pass@123" }	{ "refresh": "...", "access": "...", "role": "STUDENT", "force_password_change": true, "user": { "id": 1, "email": "user@example.com", ... } }	200, 400	Any (login)
userlogin/password/forgot/	POST	userlogin	{ "email": "user@example.com" }	{ "message": "OTP sent successfully to user@example.com" }	200, 400, 404, 500	Public
userlogin/password/verify/	POST	userlogin	{ "email": "user@example.com", "otp": "123456", "new_password": "NewPass@123" }	{ "message": "Password reset successful" }	200, 400, 404	Public (OTP holder)
userlogin/password/change/	POST	userlogin	{ "old_password": "OldPass@1", "new_password": "NewPass@2" }	{ "message": "Password changed successfully" }	200, 400	Logged-in user
userlogin/password/reset/	POST	userlogin	{ "identifier": "user@example.com", "new_password":	{ "message": "Password reset successful" }	200, 400, 404	is_staff / Admin

			"Temp@123" }			
userlogin/token/refresh/	POST	userlogin / JWT	{ "refresh": "<refresh_token>" }	{ "access": "<new_access_token>" }	200, 401	Any logged-in

7.2. Academics Setup (Module: academics)

Departments / Semesters / AY / Schema

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
academics/departments/	GET	academics	–	[{ "id": 1, "name": "CSE" }, ...]	200	Anyone
academics/departments/	POST	academics	{ "name": "CSE" }	{ "id": 1, "name": "CSE" }	201, 400	Admin
academics/departments/<id>/	GET/PUT/DELETE	academics	{ "name": "Updated" }	{ "id": 1, "name": "Updated" }	200, 204, 400, 404	Admin
academics/semesters/	GET	academics	–	[{ "id": 1, "sem_number": 1 }, ...]	200	Anyone
academics/academic-years/	GET	academics	–	[{ "id": 1, "year_name": "2025-2026" }]	200	Authenticated
academics/schemas/	GET	academics	?department=<id> (optional)	[{ "id": 1, "name": "CSE-2025", "department": 1 }]	200	Anyone
academics/schemas/	POST	academics	{ "name": "CSE-2025", "department": 1 }	{ "id": 1, ... }	201, 400	Authenticated

7.3. Student Management (Module: students)

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
students/create/	POST	students	{ "name": "Darshan", "email": "darshan@example.com", "mobile_no": "9876543210", "department_id": 1, "semester_id": 1, "academic_year_id": 2, "dob": "2004-01-01", "date_of_admission": "2025-08-01", "seat_type": "general_merit", ... }	{ "id": 10, "admission_id": "GECC000010", "message": "Registration successful. Your Admission ID is GECC000010." }	201, 400	Public (self-registration)
students/list/	GET	students	–	[{ "id": 10, "name": "Darshan", "department": 1, ... }]	200	Authenticated (HOD sees their dept only)
students/<id>/	GET/PUT	students	{ "status": "active", "semester_id": 2 }	Updated student JSON	200, 400, 404	HOD/Staff, Student (limited)
students/<id>/assign-usn/	PUT	students	{ "usn": "1GE21CS001" }	{ "status": "USN assigned" }	200, 400, 404	HOD / Exam Office
students/<id>/verify/	PUT	students	{}	{ "status": "Student registration verified by department" }	200, 400, 404	Department Staff / HOD

students/<id>/approve/	PUT	students	{ "action": "approve" }	{ "status": "Student approved and account activated" }	200, 400, 404	Principal
students/<id>/approve/	PUT	students	{ "action": "reject" }	{ "status": "Student registration rejected by principal" }	200, 400	Principal
students/me/	GET	students	–	Logged-in student's full profile	200, 403	Student
students/<id>/promote/	PUT	students	{}	{ "message": "Student promoted from semester 3 to 4." }	200, 400, 404, 500	HOD

7.4. Faculty Management (Module: faculty)

Base path: faculty/faculties/ (DRF router)

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
faculty/faculties/	GET	faculty	–	[{ "id": 1, "faculty_id": "FAC001", "name": "ABC", ... }]	200	Admin / HOD / Principal
faculty/faculties/	POST	faculty	{ "faculty_id": "FAC001", "name": "ABC", "phone": "9xx", "email": "f@x.com", "department": 1, "role": "PROFESSOR" }	Created faculty JSON	201, 400	Admin
faculty/faculties/<id>/	GET/PUT/PATCH/DELETE	faculty	{ "role": "HOD" }	Updated/Deleted	200, 204, 400, 404	Admin / Principal

Signals auto-create user + send password email when status="approved".

7.5. HOD Assignment (Module: hod_assignment)

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
hod_assignment/current/	GET	hod_assignment	–	[{ "faculty": 3, "faculty_name": "ABC", "department_name": "CSE", "assigned_date": "2025-01-01",”end_date": null }]	200	Anyone
hod_assignment/previous/	GET	hod_assignment	–	Past HOD records	200	Anyone
hod_assignment/assign/	POST	hod_assignment	{ "faculty": 3, "department": 1 }	{ "id": 5, "faculty": 3, "department": 1,"assigned_date": "2025-01-10", "end_date": null }	201, 400	Admin (IsAdminUser)

7. 6. Subject & Assignment Management (Module: subjects)

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
subjects/addSubject/	POST	subjects	{ "schema": 1, "department": 1, "semester": 3, "subject_name": "Data Structures", "subject_code": "21CS32" }	Subject JSON	201, 400	HOD / Admin
subjects/getSubjects/	GET	subjects	?department_id=1&scheme=1&semester=3	[{ "id": 10, "code": "21CS32", "name": "Data Structures" }]	200, 404	Authenticated
subjects/assign_subjects/	POST	subjects	{ "department_id": 1, "assignments": [	{ "message": "Subjects assigned	200, 400	HOD

			{ "subject_code": "21CS32", "faculty_id": 5, "semester": 3, "academic_year": 2 }] }	successfully! " }		
subjects/getAssignedProfessors/	GET	subjects	?semester=3&department_id=1&academic_year=2	[{ "subject_code": "21CS32", "faculty_id": 5, "professor_name": "ABC" }]	200, 404	HOD / Faculty
subjects/view-assigned-by-department/	GET	subjects	?department_id=1	List of all faculty-subject mappings	200, 400	HOD / Admin
subjects/professor/subjects/	GET	subjects	?faculty_id=5	List of subjects assigned to that faculty	200, 404	Faculty / HOD

7.7. Timetable Management (Module: timetable)

Base path: timetable/timetables/ (DRF ViewSet)

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
timetable/timetables/	POST	timetable	{ "department": 1, "semester": 3,"academic_year": 2, "slots": [{ "day": "MON", "start_time": "09:00", "end_time": "10:00", "assignment": 12 }] }	Created timetable with slots	201, 400	HOD
timetable/timetables/	GET	timetable	–	List of all timetables	200	Authenticated

				(filtered by role in UI)		
timetable/timetables/assignments/	GET	timetable	?semester=3&academic_year=2025-2026	{ "id": 12, "subject_code": "21CS32", "faculty_name": "ABC", ... }	200	HOD
timetable/timetables/<id>/approve/	POST	timetable	{ "action": "Approved", "remarks": "OK", "override": false }	{ "message": "Timetable approved and activated." }	200, 400, 409	Principal
timetable/timetables/pending/	GET	timetable	–	List of timetables with status "Pending"	200	Principal
timetable/timetables/faculty/	GET	timetable	– (uses logged-in faculty)	Timetables for that faculty only	200, 403	Faculty
timetable/timetables/student/	GET	timetable	–	Active timetable for student's dept/sem/AY	200, 403	Student
timetable/timetables/hod/	GET	timetable	?semester=3&academic_year=2	Timetables for HOD’s department	200, 403	HOD

7.8. IA Assessment (Module: ia_assessment)

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
ia_assessment/ia-tests/create/	POST	ia_assessment	{ "subject": 10, "ia_type": "Test1", "ia_name": "Unit Test 1", "max_marks": 25 }	IATestSetup JSON	201, 400	Faculty
ia_assessment/faculty/active-students/	GET	ia_assessment	?subject_id=10	[{ "id": 1, "name": "Std A", "usn":	200, 400, 404	Faculty

				"1GE21CS001" }, ...]		
ia_assessment/ia-tests/<test_id>/update-marks/	PUT	ia_assessment	{ "marks": [{ "student_id": 1, "obtained_marks": 20 }, { "student_id": 2, "obtained_marks": 22 }] }	{ "message": "Marks updated successfully" }	200, 400, 404	Faculty (owner of test)
ia_assessment/faculty/ia-marks/	GET	ia_assessment	?subject_id=10	List of IAMark with IA name, type, marks	200	Faculty
ia_assessment/student/ia-marks/	GET	ia_assessment	–	IA marks for logged-in student	200, 403	Student

7.9. Attendance (Module: attendance)

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
attendance/session/add/	POST	attendance	{ "subject": 10, "faculty": 5, "date": "2025-08-01", "session_count": 1 }	Session JSON	201, 400	Faculty
attendance/record/add/	POST	attendance	{ "session": 3, "student": 10, "status": "Present" }	Record JSON	201, 400	Faculty
attendance/report/	GET	attendance	?subject_id=10	[{ "usn": "1GE21CS001", "total": 20, "present": 18, "percentage": 90.0 }, ...]	200, 400	Faculty, HOD
attendance/student/<student_id>/	GET	attendance	–	{ "student_usn": "...", "attendance_percentage": 85.0, "sessions": [...] }	200, 403, 404	HOD, Faculty, Student (own via FE)
attendance/students/	GET	attendance	?subject_id=10	Active students list for that subject	200, 400, 404	Faculty

7.10. Fees & Payments (Module: fees)

Fee Structure

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
fees/fee-structures/	GET	fees	–	List of fee structures	200	Accounts, Principal, Admin
fees/fee-structures/	POST	fees	{ "department": 1, "academic_year": 2, "semester": 3, "seat_type": "gm", "title": "CSE Sem 3 GM", "components": [{ "name": "Tuition", "amount": 30000 }, { "name": "Exam", "amount": 2500 }] }	Created structure	201, 400	Accounts / Admin
fees/fee-structures/<id>/approve/	POST	fees	{}	{ "status": "approved", "assigned_to": 60, "created": 60 }	200, 403	Principal

Student Fees & Payments

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
fees/student-fees/	GET	fees	–	All student fee records (for staff)	200	Accounts / Admin
fees/student-fees/bulk-assign/	POST	fees	{ "department_id":1, "semester_id": 3,"academic_year":2,"seat_type":"gm","fee	{ "assigned_to": 60, "created": 45 }	200, 400, 403	Accounts

			_structure_id": 5 }			
fees/student-fees/my-fees/	GET	fees	–	[{ "id": 3, "semester":"3" ,"academic_year": "2025-2026","status" : "pending", "balance_amount": 32500.00 }]	200, 404	Student
fees/payments/	POST	fees	{ "student_fee": 3, "amount": 10000.00, "mode": "upi", "upi_txn_id": "TXN123", "upi_screenshot": "<file>" }	Payment JSON	201, 400	Student / Accounts
fees/payments/<id>/verify/	POST	fees	{}	{ "receipt_no": "REC-202508011230..." }	200, 403, 400	Accounts
fees/receipts/	GET	fees	?student_id=10	List of receipts	200	Accounts, Student (filtered in FE)

7.11. Grievances (Module: grievances)

Base path: grievances/grievances/

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
grievances/grievances/	POST	grievances	{ "topic": "Exam Issue","description": "Marks not updated", "target_role": "HOD" }	Grievance JSON with assigned target_user	201, 400	Student
grievances/grievances/	GET	grievances	–	Student → own grievances; Staff → grievances assigned to them	200	Student, Faculty, HOD, Principal, Accounts

grievances/grievances/<id>/	GET	grievances	–	Single grievance details	200, 404	Owner / Target User
grievances/grievances/<id>/respond/	POST	grievances	{ "response": "Issue resolved. Marks updated." }	{ "detail": "Response saved successfully." }	200, 403, 400	Target user with that role

7.12. Library Management (Module: library)

Base: library/

Books & Copies

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
library/books/	GET	library	–	List of books + copies	200	All Authenticated
library/books/	POST	library	{ "title": "OS Concepts", "authors": "Silberschatz", "department_id": 1, "mrp": 800.00 }	Created Book	201, 400	Library Officer / Admin
library/books/<id>/add_copies/	POST	library	{ "count": 5 }	{ "created_copy_ids": [1,2,3,4,5] }	201, 400	Library Officer
library/copies/	GET	library	–	List of copies	200	Library Officer

E-Books

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
library/ebooks/	POST	library	{ "book_id": 10, "file": "<pdf_file>" }	Ebook JSON	201, 400	Library Officer

Book Requests

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
library/requests/	POST	library	{ "title": "New AI Book", "authors": "XYZ", "reason": "Syllabus", "urgency": "High" }	BookRequest JSON	201, 400	Student / Faculty
library/requests/<id>/verify/	POST	library	{}	{ "status": "verified" }	200, 403	Library Officer
library/requests/<id>/final_approve/	POST	library	{}	{ "status": "purchased" }	200, 400, 403	Principal

Issue / Return / Fines

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
library/issues/	POST	library	{ "copy_id": 5,"due_date": "2025-0815" }	IssueRecord JSON	201, 400	Borrower (Student/Faculty)
library/issues/<id>/return_book/	POST	library	{}	{ "status": "returned" }	200, 400, 403	Library Officer
library/issues/<id>/mark_lost/	POST	library	{}	{ "status": "lost", "fine_id": 10 }	200, 403	Library Officer
library/fines/<id>/assign/	POST	library	{ "amount": 200.00, "reason": "Late return" }	{ "status": "assigned" }	200, 400,403	Library Officer
library/fines/<id>/mark_paid/	POST	library	{ "payment_reference": "UPI-TXN-001" }	{ "status": "paid" }	200	Borrower / Accounts / Library Officer

7.13. External Results (Module: external_results)

Endpoint	Method	Module	Request Example	Sample Response	Status Codes	Roles Allowed
external_results/	GET	external_results	–	Student → own results, Staff → all (if staff)	200	Student / HOD / Admin
external_results/	POST	external_results	{ "semester": 3, "exam_type": "FRESH", "pdf_file": "<result.pdf>" }	Created ExternalResult	201, 400	Student
external_results/<id>/	GET	external_results	–	Single result entry	200, 404	Owner / Staff
external_results/<id>/	PUT/PATCH	external_results	{ "pdf_file": "<updated.pdf>" }	Updated result + ChangeLog	200, 400	HOD / Staff (is_staff)
external_results/<id>/	DELETE	external_results	–	No content (ChangeLog keeps record)	204, 403	HOD / Staff

How to Use This as a Dev

- Use this as your **API contract** when building frontend (Flutter, React, etc.).
- For each feature, check:
    - **Endpoint path**
    - **Method**
    - **Role allowed**
    - **Expected JSON structure**

8. Testing Documentation

Testing ensures the SDMS platform is **functional, reliable, secure, scalable, and user-friendly**.

Multiple testing methodologies were used, including **unit testing, integration testing, API testing, validation testing, and user acceptance testing**.

8.1 Test Plan

Category	Description
Project Name	Student Database Management System (SDMS)
Modules Tested	User Login, Students, Faculty, Subjects, Timetable, Attendance, IA, Fees, Library, Grievance, External Results
Test Objectives	Verify correctness, security, usability, and performance of backend APIs
Test Scope	Backend REST APIs + Role-based access flows
Testing Levels	Unit Testing, Integration Testing, API Testing, Security Testing
Tools Used	Postman, Swagger/OpenAPI, Django TestCase, Python pytest, Database logs
Test Environment	Django REST Framework, PostgreSQL/MySQL, Localhost, Token Auth/JWT
Entry Criteria	All APIs developed, DB migrated, test environment configured
Exit Criteria	All prioritized test cases executed with PASS status
Test Deliverables	Test Cases Sheet, API Report, Bug Log, Final QA Signoff

Testing Guidelines

- Follow API documentation
- Validate request & response schema
- Check authorization and role protection
- Attempt invalid, missing, or incorrect payloads
- Validate DB updates after actions
- Confirm error codes are correct and meaningful

8.2 Test Case Template

Field	Description
TC-ID	Unique Test Identifier
Feature / Module	Name of the module tested
Test Description	Short explanation of test
Input / Pre-condition	Request body / required state
Test Steps	Actions performed
Expected Output	Desired result
Actual Result	Obtained output
Status	PASS / FAIL
Remarks	Error / pending fix (if any)

Sample Test Cases Table

8.2.1 User Authentication & Password Management

TC-ID	Feature	Test Description	Input	Expected Result	Status
AUTH-01	Login	Valid login with correct credentials	{email, password}	Returns access+refresh token, role	PASS
AUTH-02	Login	Invalid password	{email, wrong_pass}	Error 401, invalid credentials	PASS
AUTH-03	Forgot password	Generate OTP	{email}	Email OTP sent	PASS
AUTH-04	Reset password	Wrong OTP	{email, otp:"000000"}	Error 400	PASS
AUTH-05	Role-based security	Student accessing staff API	GET /students/list/	403 Forbidden	PASS

8.2.2 Students Module

TC-ID	Description	Input	Expected	Result
STD-01	Register new student	Valid JSON	"Registered" + admission_id	PASS
STD-02	Prevent duplicate email	Same email twice	Error 400	PASS
STD-03	HOD verify student	HOD Token → PUT	"status: verified"	PASS
STD-04	Principal approve	PUT action="approve"	"Account activated"	PASS
STD-05	Student view profile	GET /students/me/	Correct student only	PASS

8.2.3 Faculty + Subject Assignment

TC-ID	Description	Input	Expected	Status
FAC-01	Create faculty	Valid JSON	Faculty Created	PASS
FAC-02	Auto user created	Faculty added w/ email	Login creds mailed	PASS
SUB-01	Add subject	Valid payload	Created	PASS
SUB-02	Assign subject to faculty	Valid JSON	Stored & viewable	PASS
SUB-03	Faculty sees their subjects	GET w/ role=FACULTY	Filtered subjects	PASS

8.2.4 Timetable Management

TC-ID	Description	Result
TT-01	HOD creates timetable	Created w/ slots
TT-02	Principal approves	Status → APPROVED, active
TT-03	Student views timetable	Based on dept/sem/AY
TT-04	Duplicate version	Should auto-increment

8.2.5 Attendance

TC-ID	Description	Expected
ATT-01	Create session	Session saved
ATT-02	Mark attendance	Present/Absent updated
ATT-03	Attendance report	Aggregated results

8.2.6 IA Assessment

TC-ID	Description	Expected
IA-01	Create IA test	Valid JSON saved
IA-02	Update marks	Bulk update success
IA-03	Student IA view	Own marks only

8.2.7 Fees Module

TC-ID	Description	Expected
FEE-01	Create fee structure	Stored w/ components
FEE-02	Bulk assign fees	Students fees created
FEE-03	Make payment	Entry + pending verify

FEE-04	Approve payment	Receipt generated

8.2.8 Library Module

TC-ID	Description	Expected	Status
LIB-01	Add new book	Auto total copies=0	PASS
LIB-02	Add physical copies	Auto barcode generate	PASS
LIB-03	Issue book	Change copy status	PASS
LIB-04	Lost book	Auto fine = MRP	PASS

8.2.9 Grievances Module

TC-ID	Description	Expected
GRI-01	Student creates grievance	Assigned user auto
GRI-02	Staff responds	Status → Resolved

8.3 API Testing Summary

Tool	Usage
Postman	Manual CRUD validation, Auth tests, role-based flow
cURL	Debugging
Browser / Swagger	Quick checks

Key checks performed:

- Correct HTTP response codes
- JWT expiration & rejection
- Access only by authorized roles
- Input sanitization & validation

- SQL injection attempts blocked
- Tampered ID access denied
- File upload restrictions tested

8.4 Unit Testing Summary

Framework	**Files**
Python unittest / Django TestCase	tests.py inside every module
Coverage Target	≥ 80% (recommended)
Priority Tests	Authentication, Fees, Timetable, Library issuing

Example unit test stub:

```
from django.test import TestCase
from django.urls import reverse
from rest_framework.test import APIClient
class AuthTest(TestCase):
    def setUp(self):
        self.client = APIClient()
    def test_login_invalid(self):
        response = self.client.post('/userlogin/login/', {"identifier": "test", "password": "wrong"})
        self.assertEqual(response.status_code, 401)
```

9.Deployment & Setup Guide

This section describes the required prerequisites, installation steps, configuration, migration, running procedure, and backup/restoration instructions for the SDMS (Student Data Management System).

9.1 Prerequisites

Category	**Requirement**
Operating System	Windows / Linux / macOS
Programming Language	Python 3.10+
Framework	Django 4+ / DRF
Database Server	PostgreSQL 13+
Package Manager	pip
Virtual Environment	venv / virtualenv
Tools	Postman, Git, pgAdmin, Code Editor (VS Code)
Optional	Docker, Nginx/Gunicorn (Production)

9.2 Project Folder Tree

```
SDMS/
├── SDMS/                # Main Django project
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── userlogin/
├── students/
├── academics/
├── faculty/
├── subjects/
├── timetable/
```

```
├── attendance/
├── ia_assessment/
├── library/
├── grievances/
├── external_results/
├── fees/
├── templates/
├── static/
├── media/
├── env/                    # virtual environment (optional)
├── requirements.txt
├── manage.py
└── README.md
```

9.3 Environment Variables (.env)

Create a .env file inside your project root:

```
SECRET_KEY=<your_secret_key>
DEBUG=True
DATABASE_NAME=sdms_db
DATABASE_USER=postgres
DATABASE_PASSWORD=root
DATABASE_HOST=localhost
DATABASE_PORT=5432


EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_HOST_USER=your_email@gmail.com
EMAIL_HOST_PASSWORD=your_app_password
EMAIL_USE_TLS=True
```

Update settings.py to load .env using python-dotenv if required.

9.4 Install Dependencies

pip install -r requirements.txt

If requirements file missing, install manually:

pip install django djangorestframework psycopg2 pillow python-dotenv PyMuPDF django-filter

9.5 Database Configuration + Migration

9.5.1 Create database in PostgreSQL:

CREATE DATABASE sdms_db;

9.5.2 Apply migrations:

python manage.py makemigrations

python manage.py migrate

9.5.3 Create admin user:

python manage.py createsuperuser

9.6 Running the Backend Server

python manage.py runserver

Default URL:
http://127.0.0.1:8000/

9.7 Backup Database

PostgreSQL Backup:

pg_dump -U postgres -d sdms_db > sdms_backup.sql

Restore Backup:

psql -U postgres -d sdms_db < sdms_backup.sql

10.Conclusion

The **Student Data Management System (SDMS)** is designed as a **modular, secure, scalable, and role-based** academic data management platform for colleges and universities.
It eliminates manual work, reduces errors, enhances transparency, and provides centralized storage with controlled access.

The implementation includes:

✓ Secure authentication and JWT-based access

✓ Fine-grained role-based permission handling

✓ High modularity across 14 sub-apps

✓ Automated workflows for academics, library, grievance, fees & exam data

✓ REST API support for mobile & web integration

Overall, SDMS meets the defined problem objectives and ensures real-world practicality.